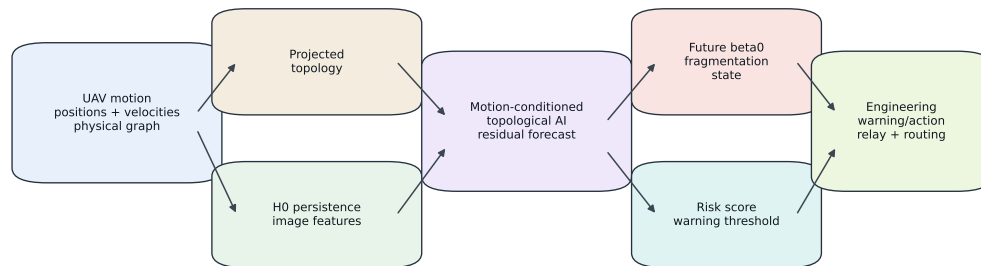


Graphical Abstract

Motion-Conditioned Topological AI for Fragmentation Risk Forecasting in UAV Networks

Ercan Erkalkan



Goal: short-horizon, interpretable fragmentation-risk forecasting for proactive UAV-network connectivity management

Validation outputs: implemented baselines, MAE/R2, risk-F1, lead-time, statistical tests, and network-level metrics

Highlights

Motion-Conditioned Topological AI for Fragmentation Risk Forecasting in UAV Networks

Ercan Erkalkan

- Topological AI forecasts UAV-network fragmentation risk.
- Motion projection improves short-horizon topology encoding.
- Persistence features support interpretable global-risk cues.
- Forecast conversion turns connectivity estimates into warning scores.
- Experiments test forecasting, risk detection, and lead time.

Motion-Conditioned Topological AI for Fragmentation Risk Forecasting in UAV Networks

Ercan Erkalkan^{a,*}

^aVocational School of Technical Sciences, Department of Electronics and Automation, Artificial Intelligence Operator Program, Marmara University, Mehmet Genç Campus, 34865 Kartal Istanbul, Turkey

Abstract

Unmanned aerial vehicle (UAV) networks can fragment within short control horizons because motion changes pairwise distance, radio margin, and multi-hop component structure. The AI contribution of this paper is a motion-conditioned topological learning framework that combines horizon projection, persistence-image descriptors, residual forecasting, and calibrated risk scoring for short-horizon fragmentation prediction. The engineering application is early warning of connectivity fragmentation in UAV networked systems to support proactive relay, routing, and delay-tolerant connectivity management. The proposed framework, Kinetic-TopoGuard, forecasts the future zeroth Betti number (β_0), converts that forecast into a calibrated fragmentation-risk score, and exposes interpretable topological and kinematic cues for decision support. A reproducible simulation benchmark evaluates fixed/adaptive topology-control policies, nominal/degraded radio conditions, tabular and graph-learning baselines, temporal neural baselines, risk-detection summaries, and early-warning lead-time outputs. The current executable evidence supports a bounded conclusion: motion-conditioned topological AI improves selected learned forecasting and risk-detection metrics under the tested synthetic benchmark, while broader engineering generalisation requires the expanded public benchmark and independent/real UAV validation protocol specified with the release.

Keywords: Topological AI, Persistent homology, UAV networks, Fragmentation-risk forecasting, Calibration, Engineering decision support

*Corresponding author. E-mail: ercan.erkalkan@marmara.edu.tr

1. Introduction

Flying ad hoc networks (FANETs) consist of swarms of unmanned aerial vehicles (UAVs) that form self-organising wireless links without fixed ground infrastructure. Individual UAVs typically travel at speeds of tens of metres per second, so inter-vehicle distances and the communication graph evolve rapidly [1]. Mission-critical applications such as post-disaster imagery aggregation, precision agriculture, and cooperative surveillance cannot tolerate unexpected partitions. Controllers therefore require fragmentation predictors that react on a sub-second timescale and still leave time for relay repositioning or a switch to delay-tolerant operation.

Accurate and, crucially, *early* estimation of connectivity is vital across such missions. An unforeseen partition can stall disaster-response map stitching, create data gaps that degrade crop-yield estimation, or open blind spots that compromise surveillance. In the reported evaluation, the practical question is not whether host-side inference can stay below the adopted 20 ms budget, but whether a model can convert low latency into genuinely useful warning signals before the next connectivity change.

In a FANET, mobility affects connectivity through a direct physical chain. When UAVs accelerate, turn, or spread during a mission, the pairwise distances between vehicles change; those distance changes modify received power, link margin, and the set of wireless edges that satisfy the adopted communication criterion. A single edge loss may be harmless if alternative multi-hop paths remain available, but several correlated edge losses can split the communication graph into disconnected components. Fragmentation is therefore not only a local link-prediction problem: it is a global topological event produced by the time evolution of UAV positions, radio reachability, and multi-hop path redundancy.

The zeroth Betti number (β_0), i.e., the number of connected components in a graph, provides a compact mathematical description of this fragmentation state. The condition $\beta_0 = 1$ means that every UAV can reach every other UAV through some multi-hop path, whereas $\beta_0 > 1$ indicates that the swarm has partitioned into two or more mutually disconnected groups. Classical algorithms such as breadth-first search, depth-first search, and union-find compute β_0 exactly and efficiently for a given snapshot [2, 3]. However, such snapshot diagnostics answer only whether the graph is fragmented *now*;

they do not estimate whether the current motion pattern is moving the swarm toward fragmentation at a future control horizon.

Persistent homology is useful in this setting because it describes not only the number of components at one fixed communication radius, but also how components appear and merge as the connectivity threshold is varied. The proposed pipeline uses only the zero-dimensional (H_0) filtration. Given UAV positions, pairwise distances are sorted from short to long; as the scale parameter grows, nearby UAVs first form small components and these components later merge through longer edges. The resulting merge distances summarize how much geometric margin separates the current component structure from alternative connectivity states. Components that merge only at large distance scales indicate weak inter-cluster coupling, while components that merge quickly indicate stronger spatial redundancy. Thus, H_0 persistence features provide a multi-threshold description of fragmentation vulnerability that is more informative than a single binary link graph.

This interpretation links persistent homology directly to dynamic FANET forecasting. UAV velocity projection estimates where nodes will be at the forecast horizon; the projected point cloud changes the pairwise-distance filtration; the H_0 persistence summary then encodes whether projected components are likely to remain robustly connected or split under small radius/link-margin perturbations. The released implementation extracts component merge times from the minimum spanning tree and converts the resulting persistence values into fixed-dimensional persistence images [4]. Persistence images are used here because they are continuous vector summaries, stable to moderate geometric perturbations, and straightforward to combine with kinematic and graph-statistical features in a regression pipeline.

The use of β_0 is nevertheless deliberately limited. It is an interpretable fragmentation indicator, but it is coarse: it counts how many components exist and does not by itself describe the size of the largest component, the existence of articulation-point vulnerabilities, or the spectral robustness of the graph before it breaks. For this reason, Kinetic-TopoGuard treats β_0 and its persistence-derived features as global early-warning descriptors that complement, rather than replace, kinematic link-margin summaries and graph-level statistics.

Recent graph-learning architectures such as GCN [5], GAT [6], and GraphSAGE [7] have also been applied to connectivity-related tasks in terrestrial MANETs [11] and VANETs [12], with a focus on link prediction, routing, or clustering. Topology-aware methods that integrate persistent homology, in-

cluding PH-GNN [8], PersLay [9], and PH-GAT [10], show that explicit topological signatures can support graph-level prediction. However, these studies do not explicitly connect UAV motion, horizon-level topology evolution, and β_0 persistence for short-horizon FANET fragmentation warning. Consequently, the key AI question addressed here is whether motion-conditioned topological descriptors improve calibrated fragmentation-risk forecasting relative to temporal graph-learning and tabular AI baselines. The corresponding engineering question is whether such forecasts provide actionable early-warning signals for connectivity management in UAV networked systems.

Research question: *Can motion-conditioned topological AI improve calibrated short-horizon fragmentation-risk forecasting for UAV networks compared with tabular, graph-learning, and temporal neural baselines, and does that improvement translate into useful connectivity-management signals?*

Contributions: To address this question, Kinetic-TopoGuard is proposed as an engineering-AI framework for connected-component forecasting in UAV swarms. The main contributions are summarised as follows.

1. **Engineering-AI problem formalisation.** Short-horizon β_0 estimation is formulated as an interpretable fragmentation-risk forecasting task for UAV networked systems under physically grounded yet still synthetic mobility and radio-link constraints.
2. **Motion-conditioned topological AI pipeline.** Kinetic-TopoGuard combines the current β_0 state, velocity-based horizon projection, projected link-margin and component summaries, H_0 persistence-image features, residual regression, and validation-calibrated risk scoring. The contribution is the integrated forecasting and decision-support pipeline rather than a claim that each individual component is new.
3. **Reproducible public benchmark.** A Python simulator, executable configuration files, manuscript-facing output artefacts, and an expanded multi-seed submission profile are released to support repeatable UAV fragmentation-forecasting experiments.¹
4. **Executed AI baselines and reproducible risk outputs.** The released compact pipeline evaluates topological, graph-learning, temporal, and tabular AI baselines that are implemented in the repository, including shallow machine-learning regressors, GCN, GAT, GraphSAGE,

¹Code and data: <https://github.com/ercanerkalkan/FANET-TopoGNN>

FANET-TopoGNN variants, T-GCN, STGCN, TGN, PI+MLP, union-find, and Kinetic-TopoGuard. The reported warning evidence is restricted to the released MAE, MSE, R^2 , latency, risk- F_1 , lead-time, statistical-test, and network-level metrics.

5. **Decision-support evaluation.** The β_0 predictor is embedded into a minimal connectivity-aware controller that performs relay repositioning and route reconfiguration. Controller-level connectivity, packet-delivery, delay, reroute, buffering, and relay-action metrics are reported as engineering decision-support outcomes rather than as a field-deployment claim.

From a networking perspective, Kinetic-TopoGuard is positioned as an *early-warning connectivity guard* that feeds into standard topology-control and routing mechanisms rather than as a stand-alone ML benchmark. In the considered setting, controllers must decide when to trigger relay repositioning, when to switch to delay-tolerant routing, and when to reconfigure multi-hop routes to avoid imminent partitions. The proposed β_0 estimator is therefore evaluated not only in terms of regression accuracy, but also through impact on risk detection and classical network-level metrics such as connectivity ratio, packet delivery ratio (PDR), and end-to-end delay under a simple store-and-forward routing scheme. Section 5 reports this metric-specific behaviour.

The remainder of the manuscript is organised as follows. Section 2 reviews GNN-based networking and topology-aware models. Section 3 introduces Kinetic-TopoGuard and the retained neural topological comparators. Section 4 details the simulation framework and evaluation protocol. Section 5 presents results and discusses result implications. Section 7 concludes the manuscript and outlines future research directions.

2. Related Work

Graph neural networks have become a standard tool for learning on structured data. Canonical architectures such as GCN [5], GAT [6], and GraphSAGE [7] achieve strong performance on node- and graph-level benchmarks by propagating information along graph edges. In FANETs, they have mainly been used for routing, clustering, coverage, or traffic-control tasks rather than explicit forecasting of future connected-component structure.

2.1. Graph Neural Networks in UAV Swarms

Learning-based networking for UAV swarms has attracted increasing interest. Pasandideh *et al.* [13] survey classical and data-driven FANET protocols and highlight the need for mobility-aware learning methods. At a broader networking level, Jiang *et al.* [14] review GNN-based routing optimisation across UAV, satellite, and next-generation wireless settings and identify scalability and deployment as persistent open issues. Lu *et al.* [15] cast multi-hop routing in a space–air–ground integrated network as a graph inference problem and employ a GNN to predict next-hop decisions. More recently, Song *et al.* [16] propose a GNN–PPO routing strategy for FANETs and report gains in packet-level QoS metrics. Although these approaches are effective for route selection and traffic engineering, the approaches do not reason about global fragmentation and do not predict future topology.

2.2. Reinforcement Learning Integration

Several studies couple GNNs with reinforcement learning for adaptive decision-making under uncertainty. Nemati *et al.* [17] investigate deep Q-learning for non-terrestrial network routing; the agent relies on handcrafted state descriptors and omits structural embeddings, which limits generalisation to unseen mobility patterns. Ye *et al.* [18] study graph reinforcement learning for partially observable multi-UAV communication coverage, showing that graph-structured control can improve coordination under uncertainty. Ren *et al.* [19] combine a heterogeneous GNN encoder with deep reinforcement learning for collaborative multi-target tracking. These studies optimise coverage, routing, or tracking decisions directly from the current or recent network state, but topology optimisation remains reactive: imminent partitions are not forecast explicitly, and topological invariants such as Betti numbers are not incorporated.

2.3. Topological Deep Learning

A separate line of work injects global structure into neural architectures by coupling GNNs with persistent homology. PH-GNN [8], PersLay [9], and PH-GAT [10] transform persistence diagrams into vector embeddings, such as persistence images, which are concatenated with learned node features. These studies show that explicit topological signals can complement learned graph representations on graph-level prediction tasks. Existing studies, however, focus primarily on static or slowly evolving graphs and do not evaluate

Table 1: Comparative overview of representative studies. “Topological features” denotes explicit use of algebraic-topology descriptors such as persistent homology. “Real-time β_0 prediction” denotes forecasting the zeroth Betti number under the adopted host-side sub-20 ms inference budget.

Study	Methodology	Primary Task	Topological Features	Real-time β_0 Prediction
Lu et al. [15]	Graph inference	Routing	No	No
Song et al. [16]	GNN + PPO	FANET routing	No	No
Nemati et al. [17]	Deep Q-learning + GNN	Routing	No	No
Ye et al. [18]	Graph RL	Coverage control	No	No
Tekdogan et al. [11]	GNN + RL	MANET clustering	No	No
Cheng et al. [12]	Spatio-temporal GNN	VANET link prediction	No	No
Ren et al. [19]	GNN + DRL	Collaborative tracking	No	No
Saadati et al. [20]	GNN clustering	Energy efficiency	No	No
PH-GNN [8] / PersLay [9]	GNN + PH	Graph/molecule regression	Yes	No
This work	Kinematic residual + PH + graph summaries	Real-time connectivity prediction	Yes	Yes

β_0 forecasting under the early-warning and latency constraints that matter in FANET control loops.

2.4. Mobility Prediction and Other Network Applications

Beyond UAV scenarios, GNN architectures have been adopted in a variety of wireless contexts. In dense wireless sensor networks, Saadati *et al.* [20] employ a GNN to form energy-balanced static clusters, improving lifetime in quasi-static environments without producing real-time connectivity metrics. In mobility prediction, Azadani and Boukerche [21] propose STAG, an attention-based GNN that captures asymmetric vehicle interactions for path prediction, whereas Capanema *et al.* [22] combine recurrent and graph layers to forecast future points of interest. These efforts suggest the value of hybrid GNNs in dynamic domains, yet none targets network-level topological quantities.

2.5. Identified Research Gap and Study Positioning

Table 1 compares representative studies along three criteria: support for dynamic topology, explicit use of topological invariants, and ability to predict the zeroth Betti number under the adopted host-side latency budget. None of the reviewed approaches directly addresses β_0 estimation. The present work studies that capability in a compact, fully reproducible benchmark rather than claiming a finished hardware deployment result. Given the operational importance of early fragmentation detection in UAV swarms, explicit prediction of connectivity changes remains both a theoretically relevant and practically underexplored problem.

From a machine-learning viewpoint, the final method deliberately moves beyond a plain topology-augmented GNN. Kinetic-TopoGuard keeps the

persistent-homology signal, but combines that signal with velocity-based horizon projection, projected link-margin features, the current connected-component state, and validation-calibrated risk scoring. The contribution therefore lies in a networking-specific forecasting algorithm: (i) adapting persistent-homology and kinetic margin features to future-horizon FANET connectivity forecasting, (ii) formulating β_0 prediction as an early-warning primitive for topology control and routing, and (iii) evaluating the forecasts through risk detection and controller-level connectivity, PDR, and delay metrics rather than through MAE alone.

3. Proposed Method: Kinetic-TopoGuard

Kinetic-TopoGuard is a kinematic-topological forecasting algorithm for the future number of connected components (β_0) in dynamic FANET environments. The algorithm uses the current graph state as a physically meaningful anchor, projects UAV positions to the forecast horizon using observed velocities, extracts projected link-margin and H_0 persistence features, learns a residual correction to the current β_0 , and calibrates a risk score for early-warning control. The earlier FANET-TopoGNN gated and concat models are retained in the benchmark as neural topological comparators. Figure 1 illustrates the final Kinetic-TopoGuard data flow used for the reported results.

3.1. Graph Construction from UAV Mobility

At each simulation step t , the planar positions of N UAVs are observed as $P_t = \{p_1^t, \dots, p_N^t\}$, where $p_i^t \in \mathbb{R}^2$ denotes the position of UAV i . These positions first induce a geometric candidate set

$$(i, j) \in C_t \iff \underbrace{\|p_i^t - p_j^t\|_2}_{d_{ij}(t)} \leq r_c(t), \quad (1)$$

where $r_c(t)$ denotes the fixed or adaptive geometric communication-radius policy at time t . In the reported setup, the actual undirected communication graph is the physical-radio graph $G_t = (V_t, E_t)$ obtained after applying bidirectional received-power filtering to the candidate set:

$$(i, j) \in E_t \iff (i, j) \in C_t \wedge P_{\text{rx}, i \rightarrow j}(t) \geq \Gamma_{\text{dBm}} \wedge P_{\text{rx}, j \rightarrow i}(t) \geq \Gamma_{\text{dBm}}. \quad (2)$$

Thus, the distance threshold supplies admissible candidate links, whereas E_t denotes the post-filter physical adjacency used for β_0 labels, risk targets, controller evaluation, and reported benchmark metrics.

3.1.1. Selecting an Appropriate Communication Radius r_c

The communication radius r_c reflects both physical-layer constraints and mission requirements.

- **Radio hardware and frequency constraints:** Off-the-shelf 2.4 GHz IEEE 802.11n radios with 20 dBm transmit power and 5 dBi antennas sustain line-of-sight (LOS) links up to 250–300 m [1]. Sub-GHz radios (e.g., 900 MHz) with directional antennas can extend this range to 800–1200 m [13].
- **Link-budget upper bound:** Assuming transmit power P_t , antenna gains G_t and G_r , receiver sensitivity P_{sens} , system losses L_{sys} , and a link margin M , the Friis equation yields

$$20 \log_{10} \left(\frac{4\pi r_c}{\lambda} \right) \leq P_t + G_t + G_r - P_{\text{sens}} - L_{\text{sys}} - M, \quad (3)$$

with $\lambda = c/f$. For representative values ($P_t = 20$ dBm, $G_t = G_r = 5$ dBi, $P_{\text{sens}} = -92$ dBm, $L_{\text{sys}} = 2$ dB, $M = 10$ dB, $f = 2.4$ GHz), the free-space theoretical upper bound r_c^{max} is approximately 3.1 km. The benchmark uses substantially smaller topology-control radii of 420 m to 567 m after mission constraints, fading, shadowing, and radius capping are applied.

- **Mission-level constraints:** Collision-avoidance protocols in dense swarms typically require mutual detection ranges of 150–500 m to ensure safe manoeuvring within 5 s update intervals [19]. Empirical analyses [1] indicate that maintaining a connected swarm of 20–50 UAVs under such conditions generally requires $r_c \gtrsim 300$ m.

3.1.2. Adaptive Radius Policy

To balance connectivity and spatial reuse, an adaptive-radius policy is adopted:

$$r_c(t) = \begin{cases} r_{\text{base}}, & \rho_t \leq \rho_{\text{thr}}, \\ \kappa r_{\text{base}}, & \rho_t > \rho_{\text{thr}}, \end{cases} \quad (4)$$

where the executed profile uses $r_{\text{base}} = 420$ m, ρ_t denotes the instantaneous UAV density inside the convex hull of P_t , $\rho_{\text{thr}} = 2.0 \times 10^{-5}$ UAV/m², and $\kappa = 1.35$.

This policy (i) increases r_c up to ≈ 567 m when the density criterion is triggered, (ii) allows the fixed and adaptive policies to be compared under identical mobility traces, and (iii) mimics adaptive modulation and coding schemes (MCS) in commercial radios at a coarse topology-control level.

All experiments in Section 4 consider both a fixed-radius baseline ($r_c = 420$ m) and the adaptive-radius policy.

3.2. GCN-Based Structural Encoder for Neural Comparators

This subsection defines the structural encoder used only by the retained FANET-TopoGNN neural comparators and by the standard GNN baselines. Kinetic-TopoGuard itself does not use a GCN encoder; it uses graph statistics, current- β_0 anchoring, velocity-projected summaries, and persistence-image features. For the neural comparators, local structural characteristics from G_t are extracted with a multi-layer GCN [5]. GCNs aggregate messages from each node’s neighbourhood and learn latent representations via stacked graph convolution layers:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}\right), \quad (5)$$

where

- $\tilde{A} = A + I$ denotes the adjacency matrix with self-loops,
- $\tilde{D}$ denotes the degree matrix of $\tilde{A}$,
- $H^{(l)} \in \mathbb{R}^{N \times d_l}$ denotes node embeddings at layer l ,
- $W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$ denotes a trainable weight matrix, and
- $\sigma(\cdot)$ denotes a non-linear activation, chosen as ReLU.

Two graph convolution layers are used to match the lightweight neural-comparator configuration and to keep the reproduced models within the same latency-oriented evaluation protocol. The broader interpretation of these GNN results is deferred to Section 5.7.

Initial node features $H^{(0)}$ contain the normalised 2D UAV coordinates and the velocity components scaled by the configured reference speed. After L GCN layers, a graph-level embedding $z_{\text{GCN}} \in \mathbb{R}^{d_g}$ is produced via global mean pooling:

$$z_{\text{GCN}} = \frac{1}{N} \sum_{i=1}^N H_i^{(L)}. \quad (6)$$

Mean pooling is selected due to robustness to varying graph sizes and effectiveness in graph-level regression tasks [24]. The resulting embedding is permutation-invariant and exhibits magnitude that is largely independent of N .

3.3. Topological Feature Extraction via Persistent Homology

To capture the global topological characteristics of UAV spatial distributions, persistent homology is applied to the point cloud P_t . The released implementation uses the zero-dimensional filtration induced by pairwise distances: as the scale $\epsilon \in [0, \epsilon_{\max}]$ grows, connected components merge at the edge lengths selected by the minimum spanning tree. The procedure is equivalent to the H_0 part of a Vietoris–Rips filtration, but the code does not instantiate higher-dimensional simplices.

The zero-dimensional persistence diagram $D_0(P_t)$ is computed to record birth and death times of connected components. Each point $(b_i, d_i) \in D_0(P_t)$ corresponds to a connected component that appears at scale b_i and merges at scale d_i .

The persistence diagram is subsequently mapped to a persistence image (PI), a stable and differentiable vector representation [4]. The PI is obtained by placing a weighted Gaussian kernel at each point (b_i, d_i) over a discretised grid $\mathbb{R}^{w \times h}$:

$$\text{PI}(x, y) = \sum_{(b_i, d_i) \in D_0} w(b_i, d_i) \mathcal{N}\left((x, y); (b_i, d_i), \sigma^2 I\right), \quad (7)$$

where $w(b_i, d_i)$ denotes a birth–persistence weighting function and $\mathcal{N}(\cdot; \mu, \Sigma)$ denotes a 2D Gaussian density. The resulting image is flattened into a vector $z_{\text{PI}} \in \mathbb{R}^{d_t}$.

Parameter selection.. Three hyperparameters govern the construction of each PI: the filtration cap $\epsilon_{\max}$, the Gaussian bandwidth σ , and the weighting function $w(b_i, d_i)$.

- **Filtration cap $\epsilon_{\max}$:** A value of $\epsilon_{\max} = 600$ m is used, which covers the fixed setup ($r_c = 420$ m) and the maximum adaptive radius in the executed profile ($r_c \leq 567$ m). This margin ensures that all potentially linkable UAV pairs are captured while avoiding artefacts at unrealistically large scales.

- **Kernel bandwidth σ :** Following stability considerations in [4], a 2D Gaussian kernel with $\sigma = 20$ m is employed. Values below 10 m can overfit high-frequency noise, whereas values above 30 m can smooth away useful structure. The interval $\sigma \in [10 \text{ m}, 30 \text{ m}]$ is therefore treated as a conservative operating region in the reported setup rather than as a separate robustness claim.
- **Weighting function:** Linear persistence weighting, $w(b_i, d_i) = d_i - b_i$, is used as in prior PI work [4, 9]. Long-lived components receive higher weight, whereas short bars caused by noise or transient link disruptions are down-weighted.

3.4. Kinetic-TopoGuard Forecasting Rule

For each snapshot, Kinetic-TopoGuard forms a feature vector

$$x_t = [z_{\text{PI}}(t), s_{\text{graph}}(t), s_{\text{kin}}(t, h), s_{\text{lag}}(t)], \quad (8)$$

where $z_{\text{PI}}(t)$ is the H_0 persistence image, $s_{\text{graph}}(t)$ contains graph statistics such as degree, density, largest-component ratio, and clustering, $s_{\text{kin}}(t, h)$ contains velocity-projected link-margin and component summaries at the forecast horizon, and $s_{\text{lag}}(t)$ contains one-step historical changes in β_0 , edge count, density, minimum distance, and largest-component ratio.

The kinematic branch projects positions by

$$\tilde{p}_i(t + h) = p_i(t) + h \Delta t v_i(t), \quad (9)$$

and recomputes a geometric horizon graph under the currently active communication radius. From this projected graph, the algorithm extracts $\tilde{\beta}_0(t + h)$, projected largest-component ratio, projected clustering, degree summaries, distance-margin quantiles, and the fraction of links expected to cross out of range. These quantities encode whether the current motion field is pushing the swarm towards or away from fragmentation.

The scalar forecast is a residual correction around the current connected-component count:

$$\hat{\beta}_0(t + h) = \text{clip}_{[1, \infty)} (\beta_0(t) + \alpha f_{\text{res}}(x_t)), \quad (10)$$

where f_{res} is selected from ExtraTrees, gradient boosting, and random-forest residual regressors, and the shrinkage coefficient $\alpha \in [0, 1]$ is selected on the

validation split to minimise MAE. Including $\alpha = 0$ in the validation grid prevents the learned correction from degrading below the current-topology anchor when the residual is not supported by validation evidence.

The risk score is learned separately from the scalar residual:

$$s_{\text{risk}}(t) = \lambda_1 p_{\text{cls}}(x_t) + \lambda_2 p_{\text{reg}}(\hat{\beta}_0(t + h)) + \lambda_3 p_{\text{kin}}(s_{\text{kin}}), \quad (11)$$

where p_{cls} is an ensemble probability from tree-based risk classifiers, p_{reg} maps the scalar forecast to a fragmentation probability, and p_{kin} is a deterministic kinematic hazard score from projected margins and projected component count. The weights $(\lambda_1, \lambda_2, \lambda_3)$ and threshold τ_{frag} are selected on the validation split using a criterion that favours fragmentation-risk F_1 while maintaining warning behaviour consistent with the forecast horizon. This decoupling is important: scalar β_0 accuracy, risk detection, and warning behaviour are related but not identical objectives.

3.5. Distinction from Retained FANET-TopoGNN Comparators

Kinetic-TopoGuard is the final proposed framework in this manuscript. The earlier FANET-TopoGNN concat and gated variants are retained only as neural topological comparators because they combine GCN graph embeddings with persistence-image features, but they do not implement the kinematic projection, residual anchoring, or validation-calibrated risk scoring used by Kinetic-TopoGuard. Table 2 summarises this distinction explicitly so that the retained comparator variants are not confused with the final proposed framework.

3.6. Retained FANET-TopoGNN Concat Comparator

The concat FANET-TopoGNN variant is kept as a compact comparator for the question: *what happens if GCN graph features and H_0 persistence-image features are simply joined and regressed directly?* It uses the GCN graph embedding z_{GCN} from Section 3.3 and the persistence-image embedding z_{PI} from Section 3.4, concatenates them,

$$z_{\text{concat}} = [z_{\text{GCN}} \parallel z_{\text{PI}}], \quad (12)$$

and feeds the result into the same two-layer regression head used for the neural comparators. This variant has no velocity-projected horizon graph, no residual anchoring to the current β_0 , and no separate risk-calibration layer. It therefore isolates a simple neural fusion design rather than representing the final Kinetic-TopoGuard method.

Table 2: Distinction between the proposed Kinetic-TopoGuard framework and the retained FANET-TopoGNN comparator variants.

Aspect	Kinetic-TopoGuard	Concat variant	Gated variant
Study role	Proposed final method	Comparator	Comparator
Inputs	PI + graph stats + kinematic summaries + lags	GCN embedding + PI	GCN embedding + PI
Motion horizon	Explicit UAV position projection and projected margins	None	None
Forecast rule	Residual correction around current β_0	End-to-end MLP	End-to-end MLP
Fusion logic	Feature-level residual/risk pipeline	Direct concatenation	Learned sigmoid gate
Risk output	Calibrated $s_{\text{risk}}(t)$	Thresholded scalar forecast	Thresholded scalar forecast
Interpretation	Motion-conditioned topological guard	Neural fusion ablation	Neural fusion ablation

3.7. Retained FANET-TopoGNN Gated Comparator

The gated FANET-TopoGNN variant is retained as a second comparator that tests whether a learned fusion gate is preferable to direct concatenation. It first projects the GCN and persistence-image embeddings to a shared latent dimension, computes a sigmoid gate, and forms

$$z_{\text{gate}} = g \odot \tilde{z}_{\text{GCN}} + (1 - g) \odot \tilde{z}_{\text{PI}}, \quad g = \sigma(W_g[\tilde{z}_{\text{GCN}} \parallel \tilde{z}_{\text{PI}}] + b_g). \quad (13)$$

The fused representation is then passed to a two-layer MLP trained with mean-squared error against $\beta_0(t + h)$. As with the concat variant, this model is a retained FANET-TopoGNN baseline: it evaluates adaptive neural fusion of structural and topological embeddings, but it does not use the motion-conditioned projected summaries or calibrated risk objective that define Kinetic-TopoGuard.

3.8. Use-case: Connectivity-aware topology control and routing

While Kinetic-TopoGuard is trained to forecast the zeroth Betti number β_0 and a separate fragmentation-risk score, the intended role in a FANET stack is to serve as an early-warning guard for topology control and routing. The predictor is therefore embedded into a simple controller that combines connectivity checking, relay repositioning, and route adaptation as follows.

At each simulation tick t , the controller receives the current snapshot (G_t, P_t) , the predicted number of connected components at the forecast horizon, $\hat{\beta}_0(t+h)$, and the calibrated risk score $s_{\text{risk}}(t)$. For scalar-only baselines, the released implementation converts the forecast into a binary *connectivity risk indicator*

$$R(t) = \mathbb{I}[\hat{\beta}_0(t+h) > 1],$$

which is zero when the swarm is predicted to remain fully connected at the horizon and one when a future partition is predicted. Kinetic-TopoGuard instead thresholds $s_{\text{risk}}(t)$ at a validation-selected τ_{frag} ; the scalar baselines use $\tau_{\text{frag}} = 0.5$. Whenever the resulting indicator is positive, the controller triggers a topology-control action:

- a small set of candidate relay UAVs with high betweenness centrality or degree in G_t is identified;
- geometric midpoints between emerging components are computed from P_t ; and
- one or two relays are assigned to move towards these midpoints while preserving collision constraints.

In parallel, a simple geographic store-and-forward routing scheme updates end-to-end paths using the current physical-radio graph G_t represented by the snapshot adjacency. When $\hat{\beta}_0(t+h)$ indicates an impending split, routes whose endpoints are likely to fall into different components are proactively re-routed through the selected relays or switched to a delay-tolerant mode. For controller-level metric computation, non-proactive ticks use the physical snapshot adjacency directly; proactive ticks keep that graph as the baseline and update only relay-incident candidate links under the same radio-model configuration.

The same controller is used for Kinetic-TopoGuard and the evaluated GNN controller variants (GCN, GAT, GraphSAGE, FANET-TopoGNN, and the concat ablation); only the β_0 predictor and associated risk score are replaced. The shared-controller design ensures that any gains in connectivity ratio, packet delivery ratio, or end-to-end delay can be directly attributed to the quality and timeliness of the connectivity forecasts rather than to differences in higher-level control logic.

In practical terms, the controller uses $\hat{\beta}_0(t+h)$ in three ways. First, when $R(t)$ exceeds the fragmentation threshold, one or two relay candidates among

the nodes with highest degree or betweenness centrality are temporarily designated as connectivity-maintaining UAVs. These relays increase transmit power within hardware limits and are steered towards geometric midpoints between the predicted components, effectively “pulling” clusters back together. Second, flows whose source and destination are likely to fall into different components under the current prediction are proactively re-routed through these relays before the predicted split materialises. Third, if $\hat{\beta}_0(t+h)$ suggests that no single relay can prevent an imminent split, flows associated with vulnerable pairs are switched to a delay-tolerant mode, where packets are buffered locally until connectivity is restored.

3.9. Choice of target variable and relation to networking metrics

The forecast target is the future zeroth Betti number $\beta_0(t+h)$, i.e., the number of connected components of the communication graph six simulation steps ahead in the executed profile. From a topological perspective, this choice is natural: β_0 is the primary invariant captured by the H_0 persistence diagram, and persistent homology provides stable summaries of how connected components appear and merge across distance scales. From a networking perspective, $\beta_0(t+h)$ directly answers the most critical early-warning question for many FANET missions: *is the swarm about to remain fully connected or split?* The condition $\beta_0 = 1$ is equivalent to the existence of an end-to-end path between every pair of UAVs, whereas $\beta_0 > 1$ unambiguously signals at least one partition.

Although practitioners rarely talk about Betti numbers explicitly, many familiar quantities can be expressed in terms of β_0 and the component partition. For instance, the number of disconnected nodes is $N - |C_{\max}|$, where $C_{\max}$ denotes the largest connected component; this quantity is zero whenever $\beta_0 = 1$. Conversely, a sudden increase in β_0 is accompanied by a reduction in $|C_{\max}|$ and by the appearance of isolated or small clusters that are effectively cut off from the rest of the swarm. In the proposed controller, β_0 and the calibrated risk score are therefore used as global early-warning primitives: scalar-only models use $R(t) = \mathbb{I}[\hat{\beta}_0(t+h) > 1]$, whereas Kinetic-TopoGuard uses $s_{\text{risk}}(t) \geq \tau_{\text{frag}}$ to trigger relay repositioning, link reinforcement, or a switch to delay-tolerant forwarding (Section 3.8).

Because β_0 is a coarse descriptor, it is used as the primary structural forecasting target rather than as a complete description of FANET dynamics. The scalar component count states how many connected components exist, but it does not show how large the surviving connected core is, whether

the graph is held together by one or two critical nodes, or how close a connected graph is to spectral disconnection. Complementary structural-resilience descriptors are therefore reported for diagnostic evaluation. These descriptors do not redefine the main target; instead, they make explicit which pre-fragmentation vulnerabilities are hidden by the scalar component count.

3.10. Complementary structural-resilience descriptors

For each communication graph $G_t = (V_t, E_t)$, three additional graph-level descriptors are computed alongside β_0 in the structural analysis. First, the giant-component ratio

$$g_t = \frac{|C_{\max}(G_t)|}{|V_t|} \quad (14)$$

measures the fraction of UAVs that remain mutually reachable inside the largest connected component. Unlike β_0 , it distinguishes a minor split involving one isolated UAV from a severe partition in which the swarm separates into similarly sized groups.

Second, the articulation-point ratio

$$a_t = \frac{|\mathcal{A}(G_t)|}{|V_t|} \quad (15)$$

counts the fraction of nodes whose removal would increase the number of connected components. A high value of a_t indicates that connectivity is fragile even when $\beta_0 = 1$, because one relay-like UAV may be acting as a bottleneck between two otherwise separated clusters.

Third, algebraic connectivity

$$\lambda_2(G_t) = \lambda_2(D_t - A_t) \quad (16)$$

is the second smallest eigenvalue of the graph Laplacian, with A_t and D_t denoting the adjacency and degree matrices. For an undirected graph, $\lambda_2 = 0$ when the graph is disconnected, while small positive values indicate weakly tied connected graphs. Thus, λ_2 can reveal pre-fragmentation vulnerability before β_0 changes from one to a larger value.

These descriptors enrich the evaluation in three ways. The giant-component ratio quantifies the severity of a predicted fragmentation event, articulation points identify node-level bottlenecks relevant to relay selection, and λ_2 gives a continuous spectral margin for connected snapshots. Kinetic-TopoGuard therefore remains a short-horizon β_0 forecaster, but the reported evaluation

includes these complementary descriptors to show whether future-fragmented horizons are also structurally weaker before the actual split. Extending the model to multi-task prediction of β_0 , g_t , a_t , and λ_2 is left as future work because it would require a separate loss design and a broader statistical evaluation.

3.11. Training Protocol and Hyperparameters

The executed manuscript tables use the publication-oriented compact profile `configs/publication_compact.json`. The separate `configs/paper_like_submission.js` profile expands the seed set to 20 for a larger confirmatory run while retaining only the model families and metrics implemented in the released pipeline. Aggregate metrics are computed over the released seed set used by the selected profile, with exact configuration files, per-seed logs, confidence-interval computations, and resume-cache artefacts included in the repository. Key hyperparameters are listed in Table 4.

Optimiser and learning rate.. The Adam optimiser [27] is employed with an initial learning rate of 10^{-3} and weight decay 10^{-4} .

Batching and epochs.. Training uses mini-batches of 64 snapshots and runs for up to 18 epochs. Early stopping with a patience of 5 epochs is applied based on validation loss.

Regularisation.. Dropout with probability 0.15 is applied in the learned models that expose a dropout parameter in the released pipeline.

Regression heads.. For the retained FANET-TopoGNN neural comparators, the concat or gated fusion vector feeds a two-layer MLP in the executed configuration:

$$h = \text{ReLU}(W_1 z_{\text{fuse}} + b_1), \quad \hat{\beta}_0 = W_2 h + b_2, \quad (17)$$

where z_{fuse} denotes either z_{concat} or z_{gate} and the hidden dimension is fixed to 48. For the direct concat comparator, the graph and PI embeddings are each 48-dimensional and are concatenated into a 96-dimensional vector before the same 48-dimensional hidden regression head. Kinetic-TopoGuard is trained separately as a residual tree-ensemble forecaster with a distinct validation-calibrated risk score, as specified in Section 3.5.

Table 4: Key training hyperparameters for the reported evaluation.

Parameter	Value
Optimiser	Adam
Initial learning rate	1×10^{-3}
Weight decay	1×10^{-4}
Batch size	64 snapshots
Epochs	18 (early stopping, patience 5)
Dropout	0.15
Seeds (reported compact run)	{7, 17, 27}
Seeds (expanded submission profile)	20 seeds, <code>configs/paper_like_submission.json</code>
Hidden size (MLP)	48
Temporal windows	{5}

Hardware and runtime. The recorded compact wall-clock measurement was obtained in the author-recorded local CPU environment without CUDA acceleration, whereas the distributed reproducibility environment is specified separately in `environment.yml`. The previous three-seed compact run required approximately 4,099 s (68.3 min) wall-clock time in that author-recorded environment, while mean per-snapshot inference latency for the learned models remained below the 20 ms budget used in the manuscript as an operational target. The expanded 20-seed submission profile is intentionally separated because it is a long-running confirmatory experiment rather than a smoke test.

3.12. Algorithmic Complexity

Algorithm 1 outlines the snapshot-wise construction procedure used in the released pipeline for the H_0 persistence image. The current implementation first forms the full pairwise distance matrix, then extracts H_0 death times from the minimum spanning tree, and finally rasterises the resulting persistence values onto a fixed 16×16 grid in the executed publication compact profile.

The topological feature pipeline is restricted to zero-dimensional homology: only births and merges of connected components (H_0) are tracked. Consequently, higher-dimensional simplices are not instantiated. In the released code, the dominant cost is the pairwise-distance and minimum-spanning-tree computation on each snapshot, with Kinetic-TopoGuard adding a sec-

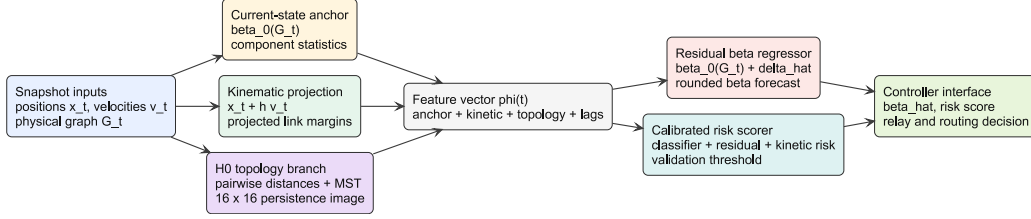


Figure 1: Kinetic-TopoGuard data flow used in the reported benchmark. The algorithm combines the current connected-component state, velocity-projected link-margin summaries, H_0 persistence-image features from pairwise distances and the minimum spanning tree, residual β_0 regression, and validation-calibrated risk scoring.

Algorithm 1 Snapshot-wise H_0 persistence image construction

- 1: **Input:** point set P_t , grid $\mathcal{G}$, bandwidth σ , filtration cap $\epsilon_{\max}$
 - 2: **Output:** PI vector $z_{\text{PI}}(t)$
 - 3: Compute the pairwise distance matrix for P_t
 - 4: Extract H_0 death times from the minimum spanning tree of the distance graph
 - 5: Clamp death times to $\epsilon_{\max}$ and map death times to persistence values
 - 6: Rasterise: for each persistence value, add a Gaussian bump on $\mathcal{G}$ with weight proportional to persistence
 - 7: Flatten $\mathcal{G}$ to obtain $z_{\text{PI}}(t)$ and return
-

ond velocity-projected distance summary and tree-based ensemble inference. This cost is acceptable for the $N \leq 30$ regime studied here but should be revisited for larger swarms. More aggressive incremental or sparsified updates are therefore better treated as future engineering extensions than as claims already established by the current implementation.

4. Experimental Setup

The simulation and evaluation framework assesses Kinetic-TopoGuard and the retained baselines. The executed pipeline is implemented in Python via `main.py` and the `fanet` package in the released repository.² Unless otherwise stated, the quantitative results reported in this manuscript correspond to the calibrated compact configuration `configs/publication_compact`.

²Repository: <https://github.com/ercanerkalkan/FANET-TopoGNN>

json; the expanded 20-seed confirmatory profile is configs/paper_like_submission.json.

4.1. Topology-Change Lead-Time Metric

The fragmentation-risk target used for classification is the exact-horizon future event $\beta_0(t+h) > 1$. It is not an any-time-within-window label: intermediate ticks between t and $t+h$ are not counted unless the forecast-horizon endpoint itself is fragmented. For the separate warning-timing analysis, we measure how early a model raises a risk warning before the next physical-adjacency change. This statistic is a topology-change warning measure, not a direct fragmentation-onset detector.

Let t denote simulation time and $\mathcal{E}(t)$ denote the physical edge set of G_t . The set of link-change event times is defined as

$$\mathcal{T}_{\text{flip}} = \left\{ \tau > 0 \mid \exists(i, j) : \mathbf{1}_{(i,j) \in \mathcal{E}(\tau^-)} \neq \mathbf{1}_{(i,j) \in \mathcal{E}(\tau^+)} \right\},$$

where τ^- and τ^+ denote left and right limits with respect to τ .

For each event $\tau_k \in \mathcal{T}_{\text{flip}}$, let $\mathcal{W}_k$ be the set of prediction times in the preceding forecast window at which the current graph is still connected and the model’s risk score exceeds the selected risk threshold:

$$\mathcal{W}_k = \left\{ t_{\text{pred}} : \tau_k - h\Delta t_{\text{sim}} \leq t_{\text{pred}} < \tau_k, \beta_0(t_{\text{pred}}) \leq 1, s_{\text{risk}}(t_{\text{pred}}) \geq \tau_{\text{frag}} \right\}.$$

The event-level warning time is then

$$\Delta t_k = \begin{cases} 0, & \mathcal{W}_k = \emptyset, \\ \tau_k - \min(\mathcal{W}_k), & \text{otherwise.} \end{cases}$$

The simulator operates on discrete time steps of length $\Delta t_{\text{sim}} = 0.1$ s. The implementation searches the preceding forecast window for each physical-adjacency flip and records the earliest qualifying warning; if no qualifying warning exists, the event contributes zero. The reported summaries include the median, interquartile range, selected percentiles, and empirical CDF of Δt_k . Interpretation of this metric is concentrated in the Early-Warning Behaviour subsection of Section 5.

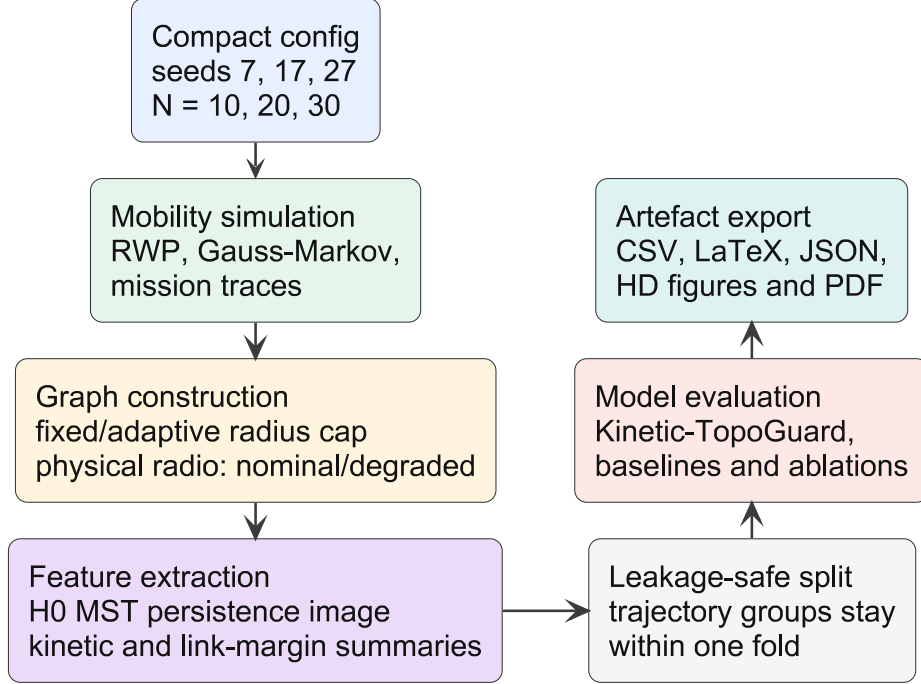


Figure 2: Executed workflow for the reproducibility-aligned evaluation. Each seed generates mobility traces, physical-radio graph snapshots, H_0 MST persistence-image features, and kinematic link-margin summaries before leakage-safe splitting, model evaluation, and manuscript-facing artefact export. Resume mode reuses completed per-model caches rather than retraining finished tasks.

4.2. Flying-Network Simulator

The released simulator generates mobility traces under Random Waypoint, Gauss–Markov, and mission-oriented motion patterns. It then constructs proximity graphs under fixed and adaptive policies, applies physical radio filtering under nominal and degraded propagation scenarios, computes H_0 persistence-image features, and exports manuscript-facing artefacts. Key configuration parameters are summarised in Table 5, and Table 6 aggregates the resulting dataset.

- **Deployment area:** A square 1 km^2 arena $(0, 1000) \times (0, 1000) \text{ m}$.
- **Swarm sizes:** $N \in \{10, 20, 30\}$ UAVs.

- **Duration and step:** Flight time $T = 20$ s with time step $\Delta t_{\text{sim}} = 0.1$ s (200 ticks per labelled run).
- **Mobility models:** Random Waypoint (RWP), Gauss–Markov (GM), and mission-oriented trajectories with run-wise random seeds.
- **Radio model:** Fixed radius $r_c = 420$ m and adaptive radius $r_c(t)$ as in (4), evaluated under nominal and degraded physical-layer scenarios.
- **Topological extraction:** H_0 distance filtration via pairwise distances and minimum-spanning-tree merge times with $\epsilon_{\text{max}} = 600$ m; persistence images of resolution 16×16 and Gaussian kernel $\sigma = 20$ m.

Each $(t, \text{run}, \text{model}, N)$ tuple yields UAV coordinates, persistence image, zeroth Betti numbers β_0^{fixed} and $\beta_0^{\text{adaptive}}$, density, and the instantaneous value of $r_c(t)$.

4.3. Task-oriented mission mobility

To complement Random Waypoint and Gauss–Markov, a task-oriented mission scenario is additionally considered. The scenario emulates typical reconnaissance, coverage, and tracking patterns reported in multi-UAV communication and coordination studies [18, 19] without assuming explicit leader–follower formations.

The 1 km^2 arena is partitioned into R rectangular mission regions (e.g., $R \in \{2, 3\}$). Each UAV is assigned to one of these regions for the duration of a run and follows a deterministic-stochastic sequence of waypoints composed of three primitive segment types:

1. *Scan legs:* straight-line trajectories that sweep the assigned region along approximately parallel lines, mimicking strip-based area coverage.
2. *Loiter points:* circular or racetrack orbits around points of interest inside the region, emulating persistent surveillance over a local area.
3. *Transit segments:* shortest-path transitions between scan legs and loiter points, or between different parts of the region.

Waypoints are generated offline for each region, and at runtime each UAV cycles through an assigned mission plan with small stochastic perturbations added to the nominal velocity and heading to model tracking errors, wind gusts, and turbulence. When a scan leg is completed or a loiter dwell time expires, the UAV transitions to the next mission segment.

Table 5: Simulation configuration (mirroring `SimulationConfig`).

Category	Parameter	Value
Arena	Size $(x_{\min}, x_{\max}, y_{\min}, y_{\max})$	$(0, 1000, 0, 1000)$ m
	Total time T	20 s
	Step Δt_{sim}	0.1 s
Swarm	UAV counts N	$\{10, 20, 30\}$
	Runs per configuration	3
Mobility	Models	RWP, GM, mission
	Speed range (RWP)	$[10, 30]$ m s ⁻¹
	Parameters (GM)	$\alpha = 0.8, \mu = 20, \sigma = 5$
	Mission regions	3
Radio	Base radius r_{base}	420 m
	Density threshold ρ_{thr}	2.0×10^{-5} UAV/m ²
	Scale factor κ	1.35
	Scenarios	nominal, degraded
PH	Filtration cap ϵ_{max}	600 m
	PI resolution	16×16 pixels
	Kernel σ	20 m
Forecast	Horizon	6 steps (0.6 s)

Compared with RWP/GM, this mission-based pattern produces connectivity dynamics that are more structured at the spatial and temporal scales of interest: UAVs tend to spend extended periods performing coordinated coverage within assigned regions, interspersed with more abrupt topology changes during transit phases or when neighbouring regions’ coverage patterns overlap. In the reported dataset, these mission traces are aggregated with the other mobility regimes rather than analysed as a separate headline result.

4.4. Train/Validation/Test Split and Leakage Control

Training, validation, and test splits are constructed using disjoint sets of random seeds and mobility initialisations to prevent temporal or spatial leakage. Assignments occur at the base trajectory level: all fixed/adaptive graph-policy and nominal/degraded radio variants from a given $(N, \text{mobility}, \text{seed})$

Table 6: Dataset summary for the reported evaluation. Each swarm size is evaluated under three mobility models, three runs per mobility/size setting, two graph policies, two radio scenarios, and 200 time steps per labelled run.

N (UAVs)	Mobility models	Runs per model	Snapshots per seed
10	RWP, GM, mission	3	7 200
20	RWP, GM, mission	3	7 200
30	RWP, GM, mission	3	7 200
Total per seed			21 600
Total across 3 compact seeds			64 800
Total across 20 expanded-profile seeds			432 000

combination are assigned to the same fold. The PI construction parameters $\epsilon_{\max}$, σ , and $w(b, d)$ are fixed by the released compact configuration, while model selection decisions that are learned from data, such as the shallow-regressor family, use the validation split. The test split remains untouched until the final evaluation. The released logs document the mapping from seeds to folds for full transparency.

Statistical-power status and expanded-seed protocol. The compact executed evidence uses three random seeds. This is sufficient for reproducibility checking, table generation, and regression diagnostics, but it is not sufficient for strong inferential claims in an applied AI study when confidence intervals overlap across several models. The release therefore includes `configs/paper_like_submission.json`, an expanded 20-seed confirmatory profile that preserves the compact mobility, graph-policy, radio-scenario, split, and horizon design while retaining the same implemented model families, including Kinetic-TopoGuard. The manuscript treats any three-seed result as preliminary unless the corresponding expanded 20-seed profile has been executed and archived. No unexecuted seed result is reported as evidence.

4.5. Physical-Layer Realism and Unidirectional Links

To approximate realistic propagation conditions, a log-distance path-loss model with log-normal shadowing is incorporated:

$$PL(d) [\text{dB}] = PL(d_0) + 10n \log_{10}(d/d_0) + X_\sigma, \quad X_\sigma \sim \mathcal{N}(0, \sigma^2),$$

where n is the path-loss exponent and σ is the shadowing standard deviation. The reported results use two radio scenarios: a nominal setting with $n = 2.35$,

$\sigma = 3.0$ dB, and receiver sensitivity -96 dBm; and a degraded setting with $n = 2.65$, $\sigma = 4.5$ dB, and receiver sensitivity -92 dBm. Small-scale fading follows a Rayleigh model (Nakagami- m with $m = 1$).

Transmit and receive chains and antenna patterns may differ across nodes, resulting in asymmetric links. Per-direction received power $P_{\text{rx}, i \rightarrow j}$ is computed, and the directed link $(i \rightarrow j)$ is considered active if $P_{\text{rx}, i \rightarrow j} \geq \Gamma_{\text{dBm}}$. An undirected edge (i, j) is introduced only if both directions satisfy the threshold. In the compact results, Γ_{dBm} is -96 dBm for the nominal scenario and -92 dBm for the degraded scenario.

Adjacency sets E_t are thus derived from received powers with fading and shadowing after the fixed/adaptive radius policy supplies the geometric candidate range. A pure distance-threshold graph remains available in the simulator as a reference construction, but the construction is not reported as a separate learning benchmark in the compact results.

Supported SINR extension.. The released simulator also supports an optional interference-limited regime for future sensitivity studies, although this mode is not activated in the reported evaluation. In that variant, time is divided into slots of length Δt_{sim} , and in each slot every node becomes a transmitter with probability p_{tx} . For a candidate directed link $(i \rightarrow j)$ active in a given slot, the instantaneous signal-to-interference-plus-noise ratio (SINR) is computed as

$$\text{SINR}_{i \rightarrow j} = \frac{P_{\text{rx}, i \rightarrow j}}{N_0 B + \sum_{k \in \mathcal{T} \setminus \{i\}} P_{\text{rx}, k \rightarrow j}},$$

where $P_{\text{rx}, i \rightarrow j}$ denotes the received power from node i at node j , $N_0 B$ is the thermal noise term, and $\mathcal{T}$ is the set of concurrently active transmitters in that slot. The directed link $(i \rightarrow j)$ is declared usable if $\text{SINR}_{i \rightarrow j} \geq \Gamma_{\text{SINR}}$, with Γ_{SINR} chosen to emulate different modulation and coding schemes. As in the power-threshold model, an undirected edge (i, j) is kept only when both directions satisfy the condition.

The interference-limited variant is included to make the simulator extensible beyond the physical received-power benchmark used in the reported evaluation. A dedicated SINR sensitivity study, however, is outside the scope of the reported evaluation and remains a natural extension for the final benchmark.

A full packet-level MAC model (CSMA/CA contention, random back-off, scheduling) is intentionally abstracted away, because the benchmark is designed to characterise the impact of rapidly changing connectivity on β_0

prediction rather than to reproduce a specific protocol implementation. The implications of this modelling scope are consolidated in Section 5.10.

4.6. Dataset Serialisation

The released reproducibility pipeline materialises snapshots in memory during each run. The pipeline exports aggregated artefacts as CSV, LaTeX, Markdown, JSON, PNG files, and vector PDF figures under the configured `outputs/` directory. The current public workspace therefore reproduces the reported tables and figures without relying on a pre-generated HDF5 bundle.

4.7. Benchmarking Persistence-Image Latency

The current pipeline records model inference latency directly in the evaluation outputs, but does not yet expose a standalone benchmark for persistence-image construction. This manuscript therefore reports end-to-end predictor latency and overall wall-clock runtime. A dedicated persistence-module micro-benchmark is deferred to future work.

4.8. Graph Construction and PyG Data Objects

For each snapshot, adjacency matrices are generated under both fixed and adaptive r_c policies and converted into the in-memory tensor and array structures used by the released training pipeline. The package also exposes an optional PyTorch Geometric adapter that converts snapshots into `Data` objects when `torch-geometric` is installed. In the reported corpus, the evaluation comprises 64,800 labelled graphs with between 10 and 30 nodes per graph. Edge counts vary with mobility, graph policy, and radio scenario; graph densities span sparse to moderately dense regimes, covering both well-connected and fragmented topologies.

4.9. Baseline Models and Training Protocol

Kinetic-TopoGuard is compared against the models executed by the released compact profile: standard static GNN baselines (GCN, GAT, GraphSAGE), the implemented tabular shallow predictors (linear regression, ridge regression, random forest, gradient boosting, and a small MLP), a purely topological model (PI + MLP), the retained FANET-TopoGNN gated and concat variants, temporal neural models with short input windows, and the union-find current-graph diagnostic reference. All reported rows operate on the same graphs and use identical train/validation/test splits.

Static GNN baselines..

- **GCN** [5]: Two GCN layers with ReLU activations and global mean pooling.
- **GAT** [6]: Two graph-attention layers with multi-head attention in the first layer and a single head in the second, followed by mean pooling.
- **GraphSAGE** [7]: Mean-aggregator variant with two graph-convolution layers and mean pooling.

Hidden dimensions, dropout rates, and learning-rate schedules are matched as closely as possible across the neural baselines. Kinetic-TopoGuard is not a neural GNN; Kinetic-TopoGuard residual and risk ensembles are selected on the same validation split, so performance differences reflect the added kinematic-topological design rather than a different test protocol.

Tabular and shallow non-GNN regression models.. Shallow snapshot-level regressors operating on hand-crafted graph and geometry features are also evaluated:

$$x_t = [\rho_t, d_{\min}(t), \bar{d}(t), d_{\max}(t), |C_{\max}(t)|, \text{avg. clustering coefficient}].$$

On top of x_t , the released shallow selector trains exactly the following candidate estimators:

- ordinary least-squares linear regression,
- ridge regression,
- random-forest regression,
- gradient-boosting regression,
- a small two-layer MLP regressor.

No additional shallow model families outside this list are reported in the released compact artefacts. Hyperparameters are fixed in the executable profile and candidate selection is performed on the validation set. Section 5 reports the selected shallow family as *Shallow ML*; per-seed output files retain the selected candidate name, whereas aggregate manuscript tables collapse the family label for readability.

Purely topological baseline: PI + MLP.. The PI+MLP baseline tests the topological branch without the GCN encoder, graph-statistical side features, or kinematic projection:

$$z_{\text{PI}} \in \mathbb{R}^{d_t} \xrightarrow{\text{MLP}} \hat{\beta}_0(t+h).$$

It is interpreted together with the shallow graph-statistical baseline, the retained FANET-TopoGNN fusion variants, and the full Kinetic-TopoGuard row in Table 11.

Temporal graph baselines.. To explicitly account for temporal evolution of the graph, the released pipeline evaluates three temporal graph-learning baselines:

- **T-GCN** (temporal GCN): a GCN followed by a GRU unit that consumes a short sequence of graphs.
- **STGCN**: a spatial-temporal GCN with temporal convolution (kernel size 3) and two computational blocks.
- **TGN**: Temporal Graph Networks with node memory and update units using the configured hidden dimension (48 in the reported configuration).

Each temporal model receives a window $G_{t-w+1:t}$ and predicts $\beta_0(t+h)$. The compact and expanded submission profiles retain the implemented $w = 5$ variants; larger temporal-window experiments remain outside the reported evidence package.

Union-find current-graph upper-bound reference. Finally, a classical union-find algorithm computes β_0 exactly from a supplied edge set. In this manuscript it is not treated as a deployable forecasting baseline. It consumes the exact connectivity graph available at the evaluated snapshot and therefore represents a current-graph diagnostic reference: it shows the error scale attainable when connectivity is already observed, not the performance of a method that predicts future topology evolution from present motion cues. For this reason, its MAE and risk- F_1 values are reported to contextualise the learned forecasters, but the learned-model ranking is discussed separately from the oracle-style reference. By construction, union-find provides no early-warning capability; a partition can only be declared once the corresponding link changes are present in the supplied graph.

Optimisation settings..

In the reported evaluation, learning-based models use Adam (10^{-3} learning rate, 10^{-4} weight decay) for up to 18 epochs with early stopping patience 5. The figures in Section 5 average over three random seeds. Per-seed metrics and resume artefacts are available in the public repository.

4.10. Evaluation Metrics

The primary regression metrics are mean absolute error (MAE), mean squared error (MSE), coefficient of determination (R^2), and average inference latency per snapshot. Fragmentation-risk detection is evaluated against the exact-horizon future target $I[\beta_0(t+h) > 1]$ using the released risk- F_1 summaries, while topology-change warning performance is assessed via the event-level lead-time distribution Δt_k defined in Section 4.1. The manuscript-level confidence intervals are reported as mean ± 1.96 standard errors across the executed seed set; per-seed output files additionally include bootstrap confidence intervals over test snapshots. In addition, the public repository includes per-run and per-seed metrics for more granular analysis.

4.11. Host-Side Inference Profiling

At present, the reproducibility suite reports per-snapshot host-side inference latency from the executed benchmark itself. Mean latency ranges from roughly 0.028 to 4.916 ms per snapshot across the learned models, with Kinetic-TopoGuard at 4.916 ms because the algorithm evaluates tree-based residual and risk ensembles. These values remain below the adopted 20 ms operational threshold in the reproduced CPU environment.

5. Results and Discussion

The calibrated evaluation uses the repository configuration named `publication_compact.json`. It covers 64,800 labelled snapshots, fixed/adaptive graph policies, and nominal/degraded radio scenarios. The results should be read as a reproducible study rather than as a claim of universal superiority.

5.1. Dataset, Graph Policy, and Radio Sensitivity

The calibrated dataset was designed to avoid the degenerate case in which adaptive radius never activates. Figure 3 shows the resulting dataset composition and graph statistics. The fixed policy uses 420 m, while the adaptive

policy averages approximately 556 m; as a result, the adaptive policy improves connectedness under both radio scenarios. Aggregated over mobility and swarm size, the adaptive/degraded setting reaches a connectivity ratio of 0.601 compared with 0.508 for fixed/degraded, and the adaptive/nominal setting reaches 0.923 compared with 0.857 for fixed/nominal. Figure 4 visualises the same policy–radio interaction as a sensitivity heat map.

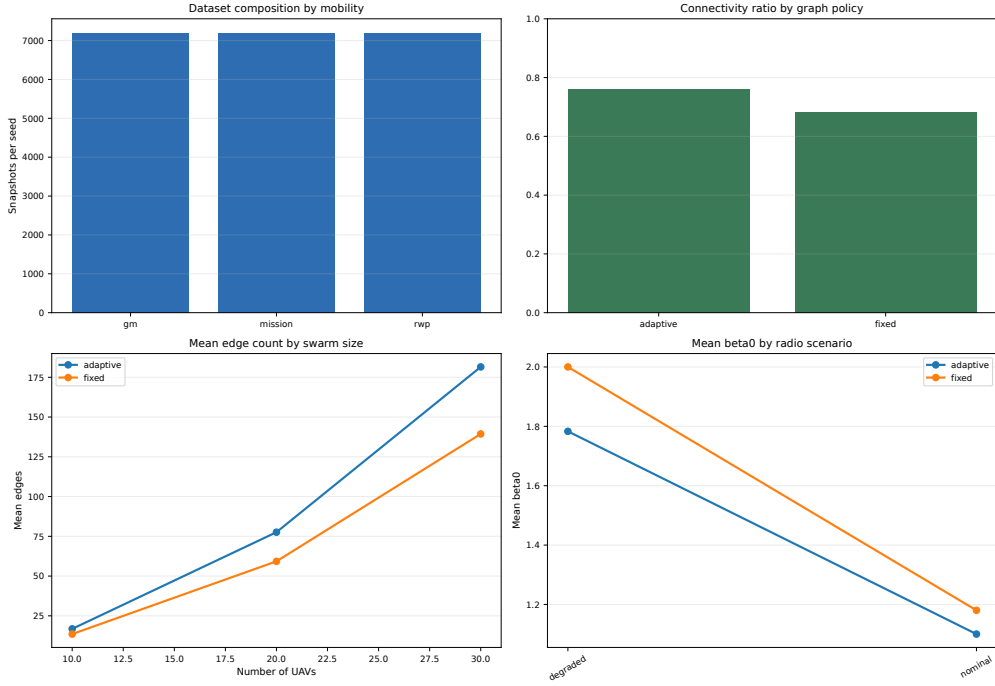


Figure 3: Dataset and graph-construction overview for the calibrated evaluation. The panels summarise mobility composition, fixed/adaptive connectivity, edge counts by swarm size, and mean β_0 under the nominal and degraded radio scenarios.

Table 7: Overall prediction performance.

Model	MAE		MSE		R2	Latency (ms)
Union-Find detection oracle	0.304	[0.282, 0.327]	0.525	[0.475, 0.576]	0.378 [0.196, 0.559]	0.204
Kinetic-TopoGuard	0.309	[0.294, 0.324]	0.404	[0.309, 0.499]	0.498 [0.205, 0.791]	4.916
Shallow ML	0.350	[0.347, 0.353]	0.373	[0.322, 0.423]	0.559 [0.427, 0.691]	0.028
GraphSAGE	0.406	[0.375, 0.437]	0.456	[0.396, 0.516]	0.464 [0.333, 0.595]	0.056
STGCN (w=5)	0.419	[0.302, 0.536]	0.490	[0.346, 0.634]	0.433 [0.365, 0.501]	0.142
T-GCN (w=5)	0.424	[0.385, 0.463]	0.475	[0.402, 0.547]	0.435 [0.286, 0.583]	0.121
GAT	0.434	[0.358, 0.511]	0.541	[0.403, 0.679]	0.372 [0.227, 0.518]	0.123
TGN (w=5)	0.441	[0.337, 0.544]	0.587	[0.428, 0.745]	0.307 [0.108, 0.507]	0.216
FANET-TopoGNN (concat)	0.445	[0.399, 0.490]	0.482	[0.412, 0.552]	0.427 [0.229, 0.625]	0.058
PI+MLP	0.450	[0.429, 0.471]	0.584	[0.431, 0.738]	0.327 [0.209, 0.445]	0.032
GCN	0.453	[0.374, 0.533]	0.554	[0.439, 0.668]	0.357 [0.226, 0.487]	0.092
FANET-TopoGNN	0.462	[0.389, 0.536]	0.503	[0.373, 0.634]	0.405 [0.178, 0.631]	0.059

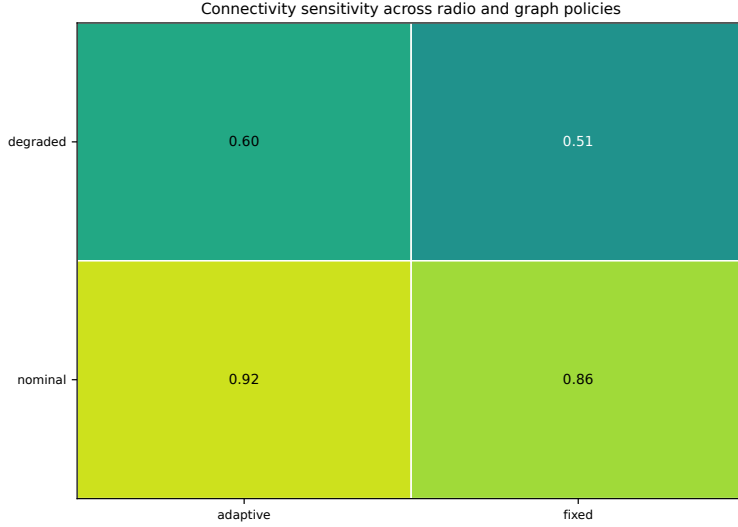


Figure 4: Connectivity sensitivity across graph policies and radio scenarios. The adaptive policy improves connectivity in both nominal and degraded propagation conditions, while the degraded radio scenario remains visibly harder.

5.2. Overall Prediction, Warning, and Network Tables

The automatically generated manuscript tables are included below without manual transcription, so the values are identical to the released **publication_compact** outputs.

Table 7 reports scalar forecasting performance. Following the reference definition in Section 3, learned models are ranked separately from the exact-graph diagnostic row. Kinetic-TopoGuard has the lowest learned MAE (0.309)

Table 8: Topology-change lead-time summary.

Model	Topology-change lead median (ms)	Topology-change lead mean (ms)	Topology-change lead 95th (ms)
FANET-TopoGNN	600.00	411.40	600.00
PI+MLP	600.00	449.27	600.00
Shallow ML	566.67	413.99	600.00
T-GCN (w=5)	566.67	411.80	600.00
GAT	533.33	394.01	600.00
GraphSAGE	533.33	370.82	600.00
STGCN (w=5)	433.33	322.01	600.00
GCN	400.00	339.88	600.00
TGN (w=5)	333.33	346.58	600.00
FANET-TopoGNN (concat)	333.33	314.70	600.00
Kinetic-TopoGuard	233.33	287.77	600.00
Union-Find detection oracle	0.00	0.00	0.00

Table 9: Network-level performance under the connectivity-aware controller.

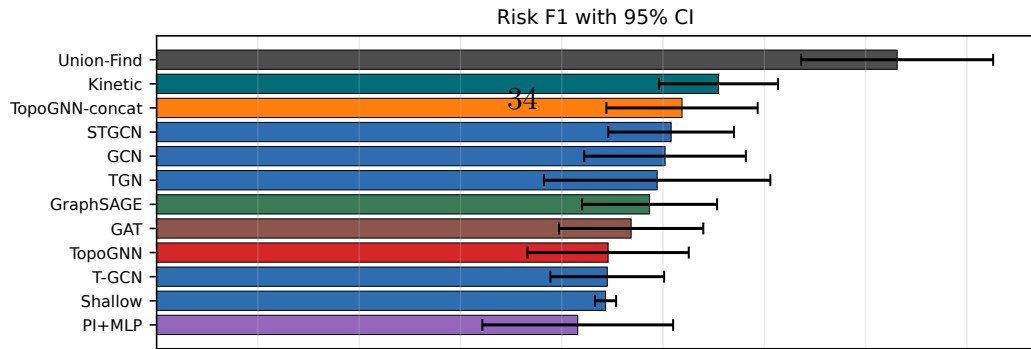
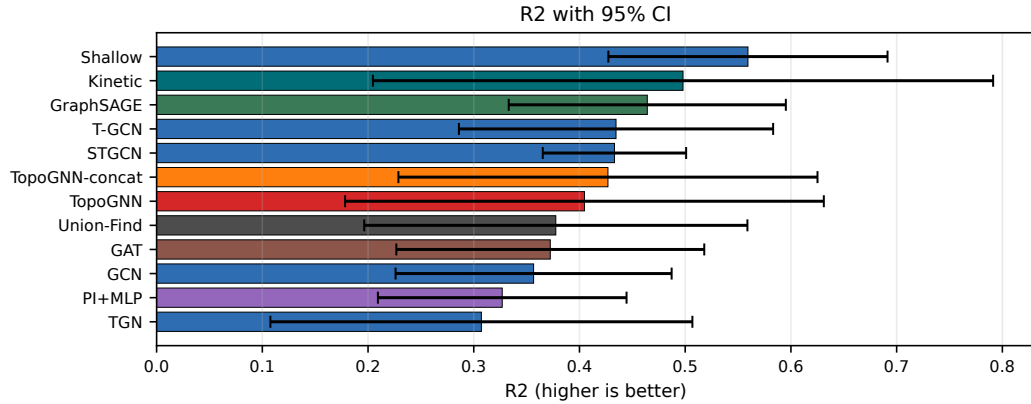
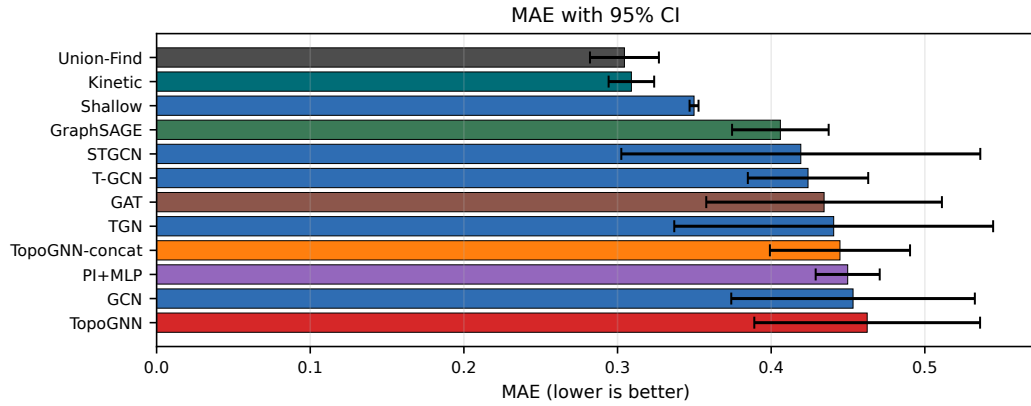
Model	Connectivity ratio	PDR (%)	Delay (ms)	Reroute (%)	DTN (%)
GraphSAGE	0.861	99.39	21.46	22.36	1.15
Kinetic-TopoGuard	0.861	99.39	21.50	18.07	1.15
GAT	0.861	99.39	21.44	26.72	1.15
GCN	0.861	99.39	21.45	23.02	1.15
FANET-TopoGNN	0.861	99.38	21.42	27.86	1.15
FANET-TopoGNN (concat)	0.860	99.37	21.46	20.62	1.14

and the strongest learned risk- F_1 (0.555); the remaining regression-fit values are reported in the same table for completeness.

Figure 5 gives the same performance comparison with 95% confidence intervals, Fig. 6 isolates the MAE ranking, and Fig. 7 shows residual distributions. A separate regression-fit-only plot is omitted because the values are already included in Table 7.

Table 10: Fragmentation-risk forecasting metrics.

Model	Risk F1	Risk Recall	Risk Precision
Union-Find detection oracle	0.731	0.738	0.724
Kinetic-TopoGuard	0.555	0.997	0.386
FANET-TopoGNN (concat)	0.519	0.972	0.355
STGCN (w=5)	0.508	0.980	0.344
GCN	0.502	0.975	0.340
TGN (w=5)	0.494	0.974	0.336
GraphSAGE	0.487	0.992	0.324
GAT	0.468	0.986	0.309
FANET-TopoGNN	0.446	0.998	0.289
T-GCN (w=5)	0.445	0.996	0.287
Shallow ML	0.443	0.972	0.287
PI+MLP	0.416	0.991	0.266



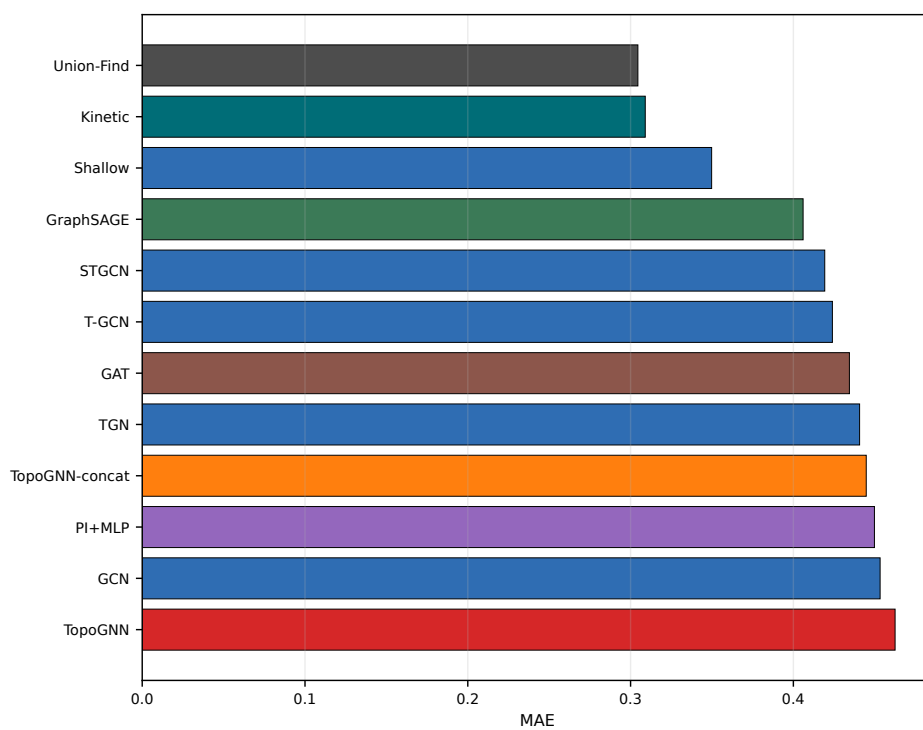


Figure 6: MAE comparison for scalar β_0 estimation across evaluated methods.

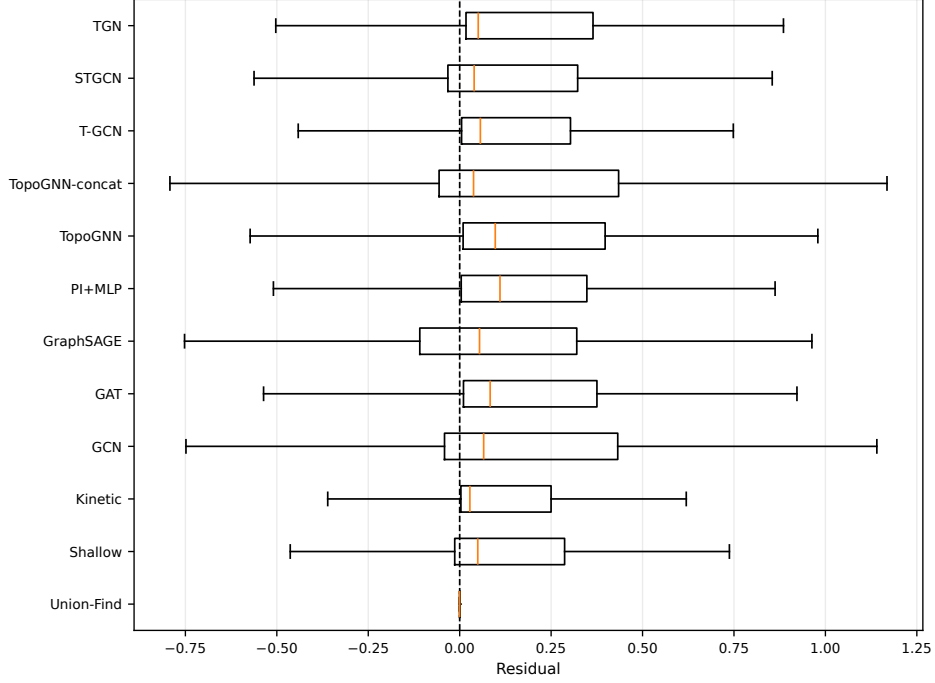


Figure 7: Residual distributions across evaluated models. The distribution confirms that the evaluation contains meaningful model differences and that topological fusion should be interpreted against strong non-deep baselines.

5.3. Early-Warning Behaviour

The forecast horizon is six simulation steps with $\Delta t_{\text{sim}} = 0.1\text{ s}$, so the maximum reported median topology-change lead is 600 ms. Table 8 and Fig. 8 show a clear ceiling effect: FANET-TopoGNN and PI+MLP reach the 600 ms median ceiling, while Shallow ML and T-GCN reach 566.67 ms. Kinetic-TopoGuard has a positive but substantially shorter median lead of 233.33 ms, indicating a more conservative warning behaviour. Its advantage, where present, is not longer warning time; it is the combination of learned MAE and learned risk F_1 in Table 10.

A positive lead is therefore treated as a minimum feasibility condition rather than as a sufficient contribution. Because long lead can be obtained by early or frequent warnings, useful early warning must be judged by timeliness and correctness together. In the current evaluation, Kinetic-TopoGuard’s risk score has very high recall but modest precision; future experiments should report precision-adjusted useful lead and lead-false-alarm trade-off

curves before claiming a decisive early-warning advantage. The empirical CDF in Fig. 9 should therefore be interpreted as a capped topology-change warning trade-off plot, not as evidence of lead-time superiority.

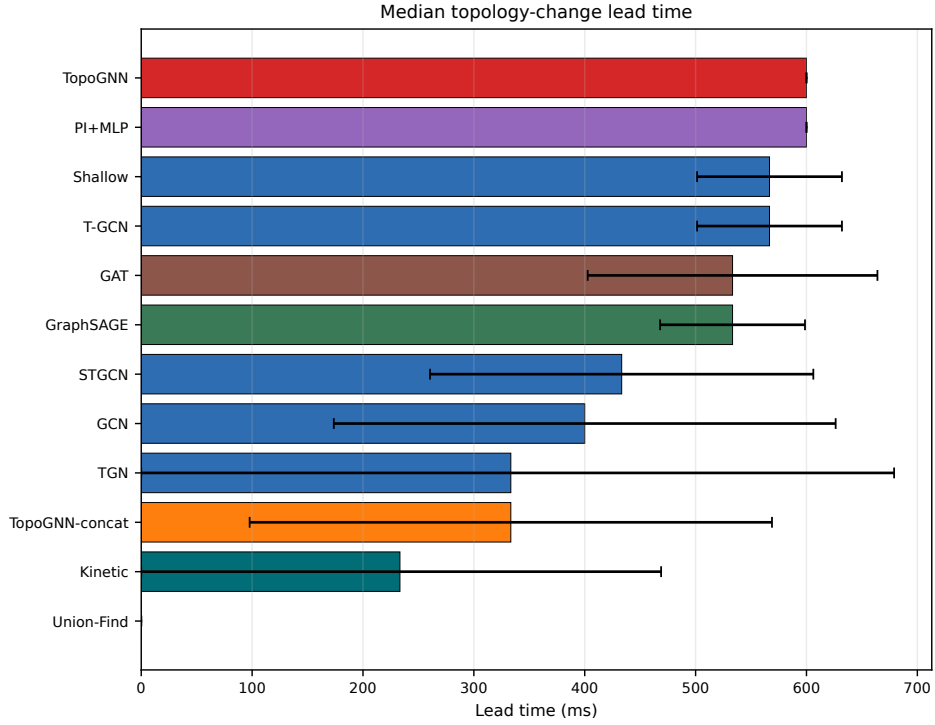


Figure 8: Median topology-change lead time with confidence intervals. Several models reach the 600 ms horizon ceiling, so lead-time claims are reported as ties where appropriate.

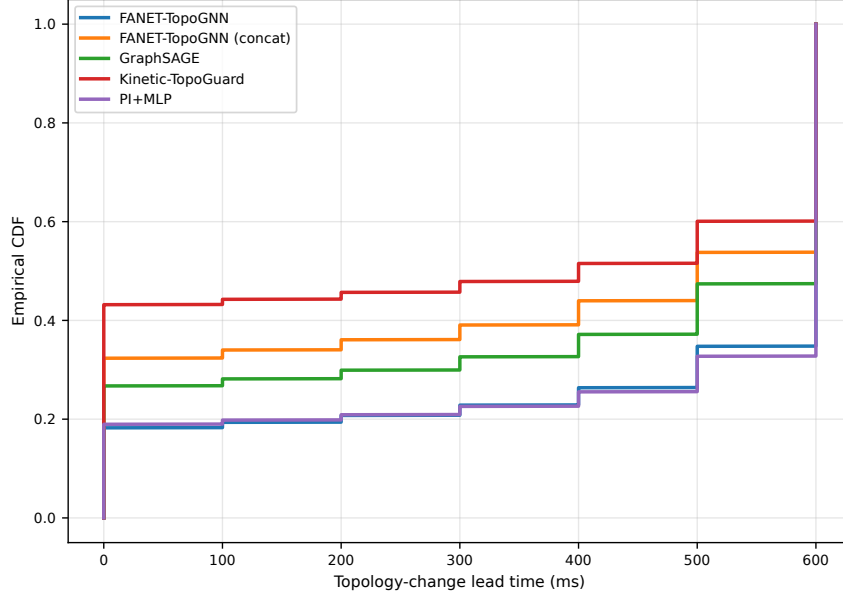


Figure 9: Empirical CDF of topology-change lead time. The curves are truncated by the six-step horizon, which motivates the cautious interpretation of lead-time differences.

5.4. Persistence-Image Feature-Source Isolation

Table 11 separates the available feature sources in the executed run. The shallow row represents graph-statistical and geometric snapshot features; PI+MLP uses the topological descriptor alone; the retained FANET-TopoGNN rows test fusion with learned GCN representations; and Kinetic-TopoGuard combines the topological branch with kinematic projection, graph-statistical summaries, residual forecasting, and calibrated risk scoring.

The results support a component-level interpretation of the topological branch within the full kinematic-topological design. A larger follow-up benchmark should still include equal-learner feature ablations over kinematic-only, graph-only, PI-only, and pairwise feature combinations.

5.5. Risk Detection, Calibration, Fusion Ablation, and Statistical Evidence

Figure 10 shows the risk- F_1 comparison. Kinetic-TopoGuard is the strongest learned risk forecaster, and Table 11 shows that PI-only prediction is not sufficient on its own. The retained fusion ablation in Fig. 11 shows that concat outperforms the adaptive gated FANET-TopoGNN variant on MAE, R^2 , and risk F_1 ; therefore no gated-fusion superiority claim is made.

Table 11: Feature-source isolation for the topological branch. Values are taken from the released evaluation artefacts. Rows compare shallow snapshot features, PI-only prediction, PI fusion with message passing, and the full Kinetic-TopoGuard design.

Evidence row	Feature source / role	MAE	R^2	Risk F_1	Latency (ms)
Shallow ML	Graph-statistical and geometric snapshot features	0.350	0.559	0.443	0.028
PI+MLP	PI features only	0.450	0.327	0.416	0.032
FANET-TopoGNN	PI fused with a GCN-style encoder	0.462	0.405	0.446	0.059
FANET-TopoGNN (concat)	PI concatenated with learned graph representation	0.445	0.427	0.519	0.058
Kinetic-TopoGuard	PI combined with graph-statistical, kinematic-projection, residual, and calibrated-risk features	0.309	0.498	0.555	4.916

The paired statistical summary in Fig. 12 reinforces the same cautious conclusion. With only three seeds, the current evidence package is useful for reproducibility checking and model diagnostics, but not for strong inferential claims in isolation. The released `stats_tests.csv` and `stats_tests.tex` files provide the exact paired tests and FDR-adjusted values for the metrics actually exported by the compact profile.

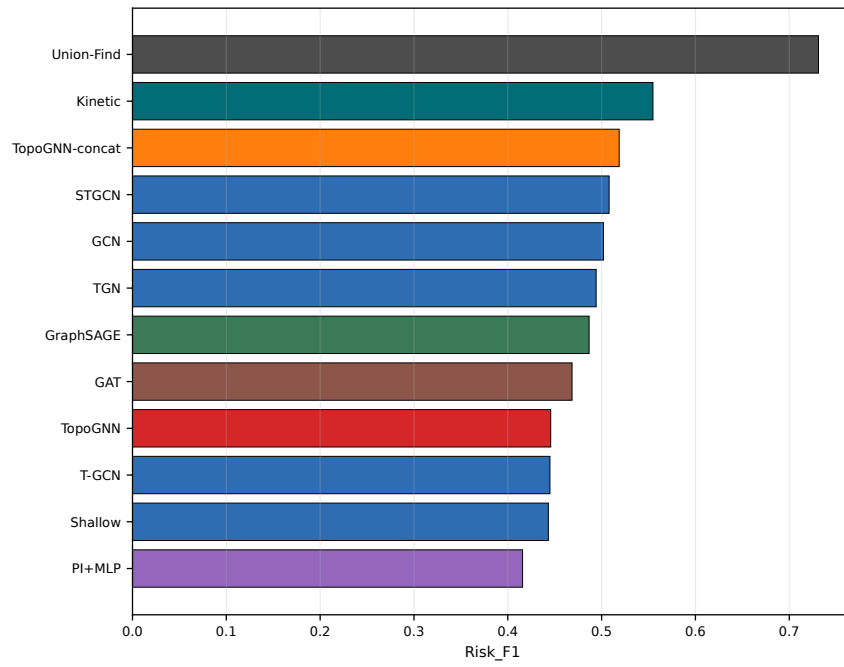


Figure 10: Fragmentation-risk F_1 comparison across evaluated methods.

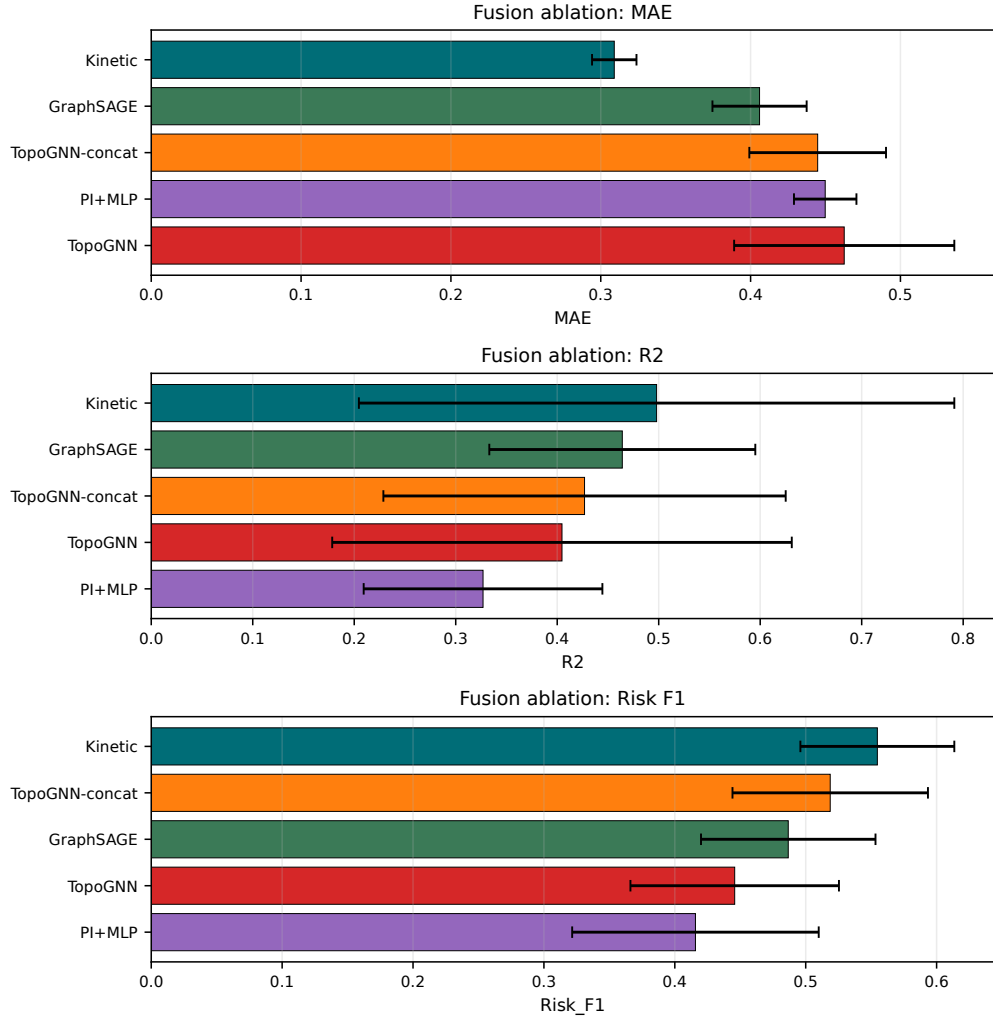


Figure 11: Fusion ablation summary. Kinetic-TopoGuard improves on both retained topological neural variants; within the legacy FANET-TopoGNN family, direct concat still outperforms adaptive gating on MAE, R^2 , and risk F_1 .

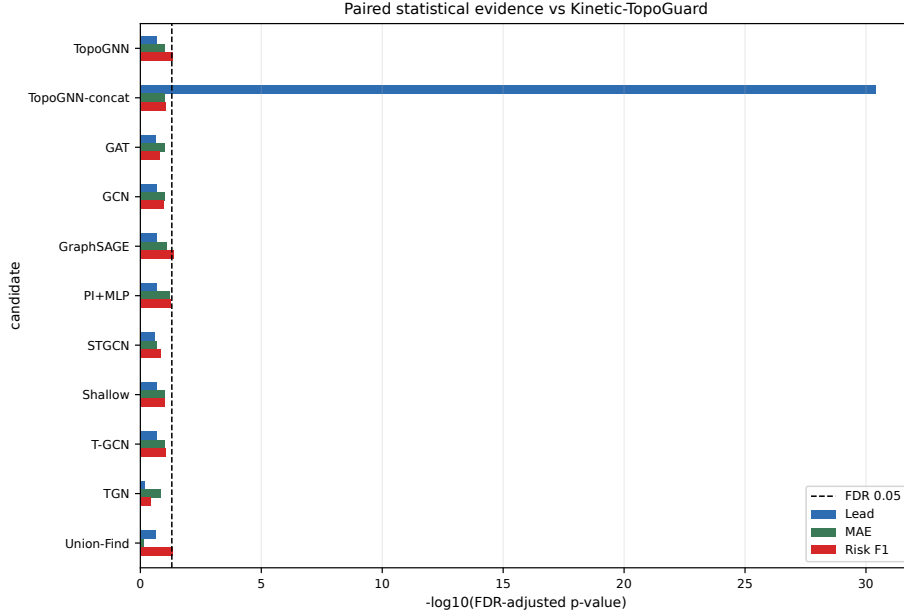


Figure 12: Paired statistical evidence relative to Kinetic-TopoGuard. The figure visualises FDR-adjusted paired-test evidence and highlights why the manuscript reports metric-specific claims rather than blanket dominance.

5.6. Complementary Structural-Resilience Diagnostics

To address the fact that β_0 gives only a coarse partition count, the reported evaluation also computes the three structural descriptors defined in Section 3.10. Table 12 reports a representative diagnostic subset sampled every tenth simulation tick from the released compact configuration. The table is not used to claim a new forecasting result; it is included to verify that snapshots whose forecast horizon ends in fragmentation also exhibit weaker structural resilience under complementary graph measures.

Table 12: Complementary structural-resilience diagnostics. Values are means over a representative every-tenth-tick subset of the released simulation profile. Future-fragmented horizons show a smaller giant connected component, more articulation-point bottlenecks, and lower algebraic connectivity.

Future state	Snapshots	Giant comp. ratio	Articulation ratio	λ_2
Connected ($\beta_0(t+h) = 1$)	1556	0.986	0.033	2.087
Fragmented ($\beta_0(t+h) > 1$)	604	0.794	0.182	0.111

The diagnostic pattern is consistent with the interpretation of β_0 as an interpretable but incomplete fragmentation signal. When the future horizon remains connected, the largest component contains almost all UAVs on average, articulation-point density is low, and the Laplacian spectral margin is positive. When the future horizon is fragmented, the giant-component ratio falls, the articulation-point ratio rises, and λ_2 approaches zero. These results do not replace the primary $\beta_0(t+h)$ forecasting task, but they enrich the networking interpretation: a controller can use β_0 for the split/no-split warning, giant-component ratio for fragmentation severity, articulation points for relay-bottleneck identification, and λ_2 as a continuous pre-fragmentation robustness margin. The manuscript therefore avoids treating β_0 as a complete description of resilience and explicitly identifies multi-metric structural forecasting as the next methodological extension.

5.7. Interpreting Message-Passing GNN Limitations

The reported results also clarify how the GNN baselines should be interpreted. Forecasting future β_0 is a graph-level topological task rather than a local node-labeling problem. A single bridge edge can determine whether the swarm remains connected, and the relevance of that bridge depends on alternative multi-hop paths elsewhere in the graph. Standard GCN, GAT, and GraphSAGE baselines observe such information only through finite-hop neighbourhood aggregation followed by graph pooling. Therefore, poor or moderate performance may arise from two related but distinct sources.

First, the evaluated static GNNs are intentionally shallow. With two message-passing layers, their receptive field can be too small to capture component-level dependencies that span several hops. This is an architectural limitation of the evaluated configuration and should not be overgeneralised to all possible GNNs. Second, even deeper message-passing architectures have an intrinsic locality bias: they repeatedly exchange information along edges and must compress increasingly distant structural evidence into node embeddings before pooling. For global fragmentation signals, this can make sparse cuts, articulation-like bridges, or long-range redundancy difficult to represent efficiently. Deeper models may reduce the receptive-field problem, but they can introduce over-smoothing, optimisation instability, and higher latency.

The results are therefore not interpreted as showing that GNNs are unsuitable for FANETs. They show a narrower point: under the tested configuration, the evaluated message-passing baselines do not capture the future

connected-component signal as effectively as a design that explicitly includes current- β_0 anchoring, velocity-projected link margins, and H_0 persistence summaries. Future work should test deeper architectures, graph transformers, positional encodings, global pooling operators, and hybrid temporal-topological GNNs before drawing a broader conclusion about the limits of neural graph models.

5.8. *Controller-Level Impact*

Table 9 reports the shared controller results after evaluating controller paths on the physical-radio snapshot graph rather than on a rebuilt geometric radius graph. Kinetic-TopoGuard and GraphSAGE attain the highest rounded connectivity ratio (0.861) and PDR (99.39%), with GAT, GCN, and FANET-TopoGNN within approximately 3×10^{-4} in connectivity ratio. The differences appear mainly in controller actions: FANET-TopoGNN has the highest proactive-reroute rate (27.86%), whereas Kinetic-TopoGuard is more conservative at 18.07% and uses fewer relay actions on average (6282 versus 7847 for FANET-TopoGNN). Figures 13 and 14 include both the extended controller metrics and the compact three-panel network summary.

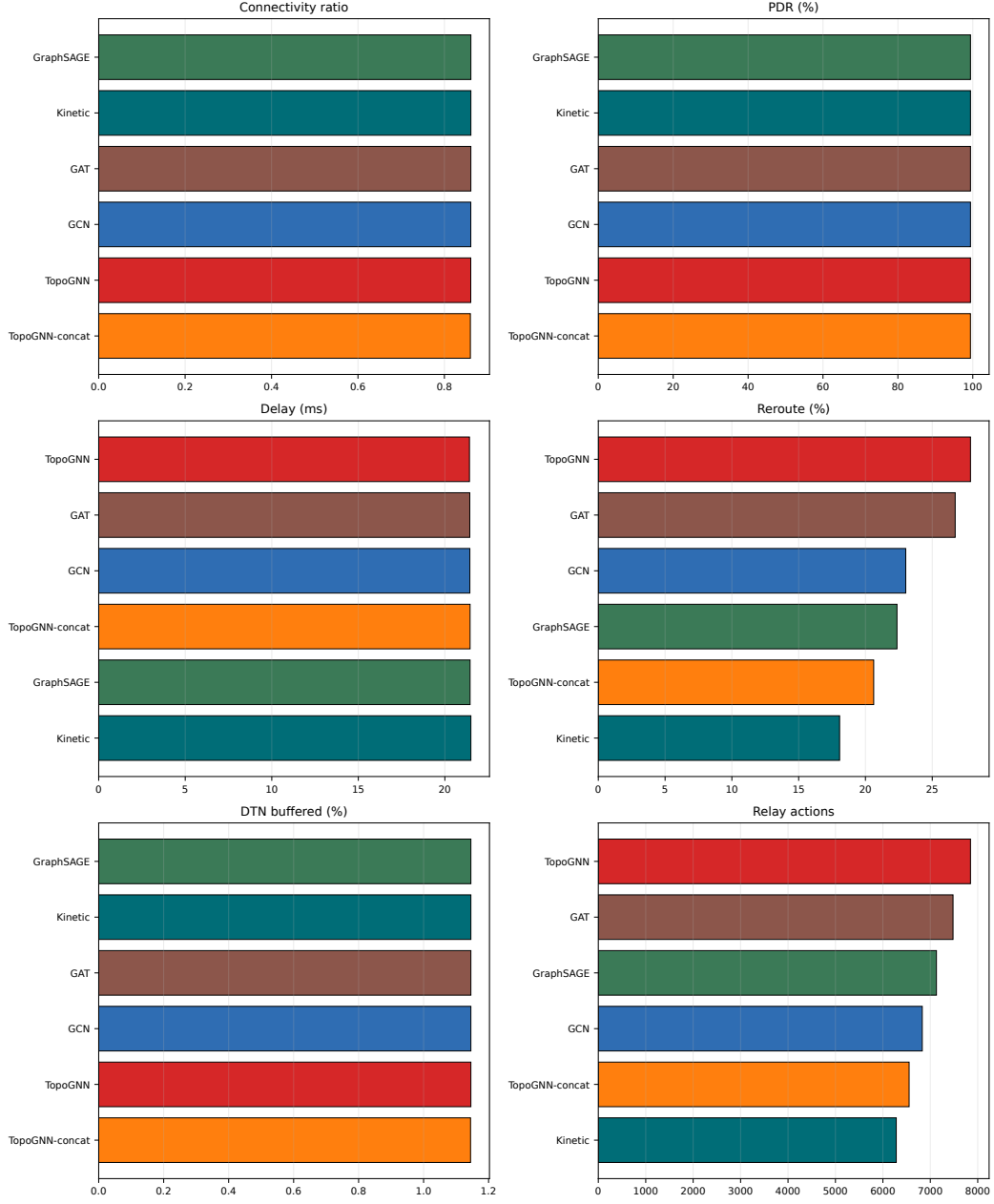


Figure 13: Extended controller-level metrics, including connectivity, PDR, delay, proactive rerouting, DTN buffering, and relay actions, computed on the physical-radio snapshot graph. Kinetic-TopoGuard matches the strongest evaluated GNN controller on connectivity and PDR while using a more conservative reroute/relay policy.

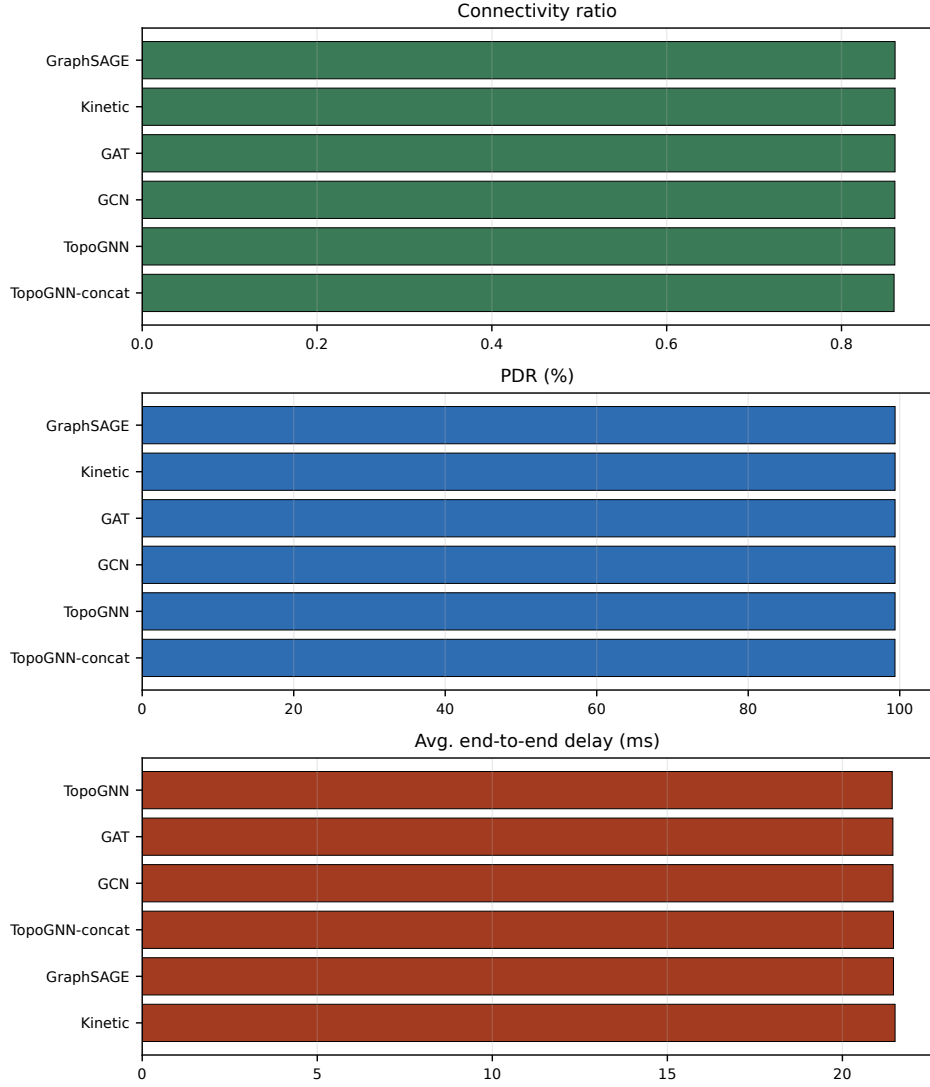


Figure 14: Controller-level summary for connectivity ratio, packet delivery ratio, and average end-to-end delay. Differences among the strongest GNN controllers are small in this calibrated benchmark.

5.9. Latency and Practical Interpretation

All learned models remain below the 20 ms online budget on the host machine used for reproduction, as shown in Fig. 15. The full calibrated run required approximately 4,099 s in the local CPU environment, while per-snapshot learned-model inference remained at or below roughly 4.916 ms on

average.

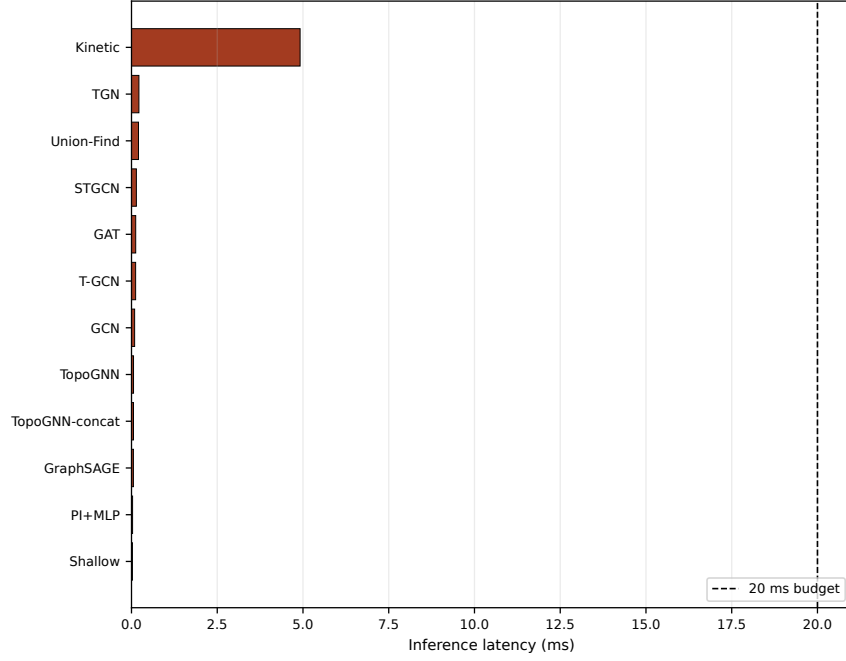


Figure 15: Inference latency relative to the 20 ms operational budget. All evaluated learned models remain well below the budget on the host machine used for the reported evaluation.

The defensible conclusion is metric-specific. Kinetic-TopoGuard is best interpreted as the strongest evaluated learned risk forecaster in this evaluation, not as a universally superior predictor.

5.10. Limitations

Several limitations remain. First, all currently reported results are produced on a synthetic simulation suite rather than on real multi-UAV flight traces, so external validity is limited. The benchmark includes path loss, shadowing, fading, and bidirectional link filtering, but the headline results do not include a full MAC model, packet-level contention, or an activated interference-limited evaluation; hardware-in-the-loop traces, real-flight validation, and onboard profiling are therefore still required before deployment-oriented claims can be made. Second, any three-seed compact result has limited statistical power and must be replaced by the released expanded multi-seed submission profile before strong inferential claims are made. Third,

the current topological branch models only H_0 and computes persistence-image features snapshot by snapshot via pairwise distances and a minimum spanning tree, so incremental updates and large-swarm scaling remain future work. Fourth, the feature-source isolation in Table 11 is not yet a complete equal-learner factorial ablation over all kinematic, graph-statistical, and persistence-image combinations. Fifth, the added giant-component ratio, articulation-point ratio, and algebraic-connectivity analysis is diagnostic rather than a full multi-task forecasting benchmark; future work should predict these descriptors jointly with β_0 and report their controller-level value. Sixth, the expanded baseline set is now supported by the code, but unexecuted optional booster or sequence results should not be discussed until their exported CSV, LaTeX, and figure artefacts are generated. Finally, the controller study uses a deliberately simple decision layer, and the resulting network-level differences among the strongest predictors are modest. A larger benchmark with more seeds, horizon sweeps, independent/public UAV mobility traces, packet-level network simulation, and tighter integration with routing or relay-control logic would be the most informative next test of generality.

6. Reproducibility

The reproducibility package available in this workspace is organised as an executable Python project rather than as a notebook-centred release. The main entry point is `main.py`. Experiment settings live in `configs/`, the end-to-end pipeline in `fanet/`, and PowerShell helpers in `scripts/`. The base environment in `environment.yml`, `requirements.txt`, and `pyproject.toml` intentionally covers the non-deep scientific stack used for dataset generation, Kinetic-TopoGuard, union-find, shallow/tabular baselines, smoke tests, and table/figure aggregation. Configurations containing neural GNN or temporal baselines require the documented `deep` extra, or `requirements-deep.txt`, before the corresponding rows should be interpreted as actual PyTorch neural results. The `pyg` extra is needed only for PyTorch Geometric snapshot export.

For the reported evaluation, the relevant command sequence is

```
python -m pip install -r requirements-deep.txt
python main.py -config configs/publication_compact.json
```

For the expanded 20-seed confirmatory run, the launch command sequence is

```
python -m pip install -r requirements-deep.txt
python main.py -config configs/paper_like_submission.json
```

If execution is interrupted, rerun the same command with the additional `--resume` flag; the pipeline verifies per-model cache coverage before skipping a seed, so interrupted or incrementally expanded studies continue from the last completed task. Each run exports manuscript-facing artefacts under the selected output directory, including `summary.json`, `runtime_profile.json`, `claims_summary.md`, `report.md`, `manuscript_tables.tex`, `tables`, and `figures`. The runtime profile separates the author-recorded runtime environment from the distributed reproducibility environment so that wall-clock claims are not conflated with the reference rerun specification. The numbers reported in this manuscript were cross-checked directly against `outputs/publication_compact/`, and the plotted figures were copied into the manuscript package under `figures/generated/`.

7. Conclusion and Future Work

Kinetic-TopoGuard has been introduced as a kinematic-topological algorithm for predicting the future number of connected components in dynamic flying ad hoc networks. The calibrated evaluation gives a metric-specific result: the method is the strongest evaluated learned model by MAE (0.309) and fragmentation-risk F_1 (0.555), while other metrics show that simpler predictors remain competitive in selected views. These results support the value of combining motion-conditioned margins, H_0 persistence summaries, graph statistics, residual regression, and risk calibration for learned warning-oriented forecasting, but they do not establish universal dominance or field-ready deployment.

The most informative next step is therefore to execute and archive the expanded 20-seed submission profile, then extend the validation campaign with horizon sweeps, public or hardware-in-the-loop flight traces, and packet-level MAC/interference studies. Beyond that, future work should incorporate richer temporal-topological hybrids and move from diagnostic structural descriptors to multi-task forecasting of β_0 , giant-component ratio, articulation-point ratio, and algebraic connectivity. This would allow future controllers to distinguish merely disconnected states from severe partitions, relay-bottleneck vulnerability, and spectrally weak but still connected formations.

Appendix A. Code and Data Availability

The public GitHub repository is ErcanErkalkan/FANET-TopoGNN at <https://github.com/ErcanErkalkan/FANET-TopoGNN>, with the archived release available at <https://doi.org/10.5281/zenodo.20226053> and the current compact consistency tag v0.1.12. It contains the executable pipeline used in this manuscript:

- `main.py`: command-line entry point for dataset generation, training, evaluation, and report export.
- `fanet/`: core package for mobility generation, topology features, datasets, models, training, evaluation, and table export.
- `configs/`: experiment configurations, including quickstart, full, compact, `publication_compact.json`, and `paper_like_submission.json` profiles.
- `scripts/`: helper scripts for environment setup and Windows execution, including resumable compact and full runs.
- `outputs/publication_compact/`: cross-checked compact artefacts used in the reported evaluation, including tables, figures, `claims_summary.md`, `report.md`, `summary.json`, `runtime_profile.json`, `stats_tests.csv`, and `artifact_manifest.txt`.
- `outputs/paper_like_submission/`: expected output directory for the expanded 20-seed confirmatory run using the model families and metrics implemented in the released pipeline.
- `environment.yml`, `requirements.txt`, and `pyproject.toml`: aligned base environment specifications for the non-deep scientific stack; `requirements-deep.txt` or the `deep` extra is required for manuscript-facing PyTorch neural baselines.

This workspace does not currently contain a monolithic HDF5 release, a notebook-only reproduction path, or a dedicated edge-profiling script, so those claims have been removed from the manuscript text.

Appendix B. Execution Notes

Representative commands for the reported benchmark are shown below:

```
python main.py -config configs/publication_compact.json
```

```
python main.py -config configs/publication_compact.json -resume
```

```
python main.py -config configs/paper_like_submission.json
```

```
powershell -ExecutionPolicy Bypass -File  
scripts\sync_manuscript_assets.ps1 -Profile publication_compact  
-ManuscriptDir ..\FANET-TopoGNN
```

```
powershell -ExecutionPolicy Bypass -File  
scripts\sync_manuscript_assets.ps1 -Profile  
paper_like_submission -ManuscriptDir ..\FANET-TopoGNN
```

The generated manuscript artefacts can be found under `outputs/publication_compact/`. The authoritative cross-check sources for the generated LaTeX tables and discussion are:

- `manuscript_tables.tex`
- `summary.json` and `runtime_profile.json`, with separate author-recorded runtime and reproducibility-environment metadata
- `metrics_overall.csv`
- `claims_summary.md`
- `lead_time_summary.csv`
- `risk_metrics.csv`
- `network_metrics.csv`
- PDF and PNG figures under the configured output directory

Appendix C. Temporal Baselines: Hyperparameters

The temporal-baseline evaluation uses T-GCN, STGCN, and TGN with window size $w = 5$, learning rate 10^{-3} , weight decay 10^{-4} , batch size 64, a maximum of 18 epochs, and early stopping with patience 5. These settings are aligned with `configs/publication_compact.json`. No window-3 or window-9 runs are used in the reported calibrated results.

Within that configuration, STGCN uses a temporal kernel size of 3, T-GCN uses a GRU-style recurrent update, and TGN uses the default memory/message dimensions implemented in the released codebase. The temporal baselines are intentionally kept simple in this run so that the manuscript can compare temporal smoothing against explicit topological features. Richer temporal-topological hybrids remain future work rather than part of the cross-checked results reported here.

CRedit authorship contribution statement

Ercan Erkalkan: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing—original draft, Writing—review & editing, Visualization.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have influenced the work reported in this article.

Data availability

All data and code are openly available from the GitHub repository [ErcanErkalkan/FANET-TopoGNN](https://github.com/ErcanErkalkan/FANET-TopoGNN) at <https://github.com/ErcanErkalkan/FANET-TopoGNN> and the archived Zenodo release at <https://doi.org/10.5281/zenodo.20226053>.

Declaration of Generative AI and AI-assisted Technologies in the Manuscript Preparation Process

During the preparation of this work, the author used OpenAI GPT-based tooling to support language refinement, local reproducibility cross-checks against generated artefacts, and maintenance of the released experimental

pipeline. After using these tools, the author reviewed, edited, and verified all manuscript content, code changes, tables, and interpretations, and takes full responsibility for the final submitted work.

References

- [1] İlker Bekmezci, O. K. Sahingoz, Şamil Temel, Flying ad-hoc networks (fanets): A survey, *Ad Hoc Networks* 11 (3) (2013) 1254–1270. doi:10.1016/j.adhoc.2012.12.004.
URL <https://www.sciencedirect.com/science/article/pii/S1570870512002193>
- [2] R. E. Tarjan, Efficiency of a good but not linear set union algorithm, *J. ACM* 22 (2) (1975) 215–225. doi:10.1145/321879.321884.
URL <https://doi.org/10.1145/321879.321884>
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 3rd Edition, MIT Press, Cambridge, MA, 2009.
URL <https://books.google.com/books?hl=en&lr=&id=RSMuEAAAQBAJ&oi=fnd&pg=PR13>
- [4] H. Adams, T. Emerson, M. Kirby, R. Neville, C. Peterson, P. Shipman, S. Chepushtanova, E. Hanson, F. Motta, L. Ziegelmeier, Persistence images: a stable vector representation of persistent homology, *J. Mach. Learn. Res.* 18 (1) (2017) 218–252.
- [5] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, *arXiv preprint arXiv:1609.02907* (2016).
- [6] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks (2018). arXiv:1710.10903.
URL <https://arxiv.org/abs/1710.10903>
- [7] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS’17*, Curran Associates Inc., Red Hook, NY, USA, 2017, p. 1025–1035.
- [8] C. Hofer, R. Kwitt, M. Niethammer, A. Uhl, Deep learning with topological signatures, in: *Proceedings of the 31st International Conference*

- on Neural Information Processing Systems, NIPS'17, Curran Associates Inc., Red Hook, NY, USA, 2017, p. 1633–1643.
- [9] M. Carrière, F. Chazal, Y. Ike, T. Lacombe, M. Royer, Y. Umeda, Perslay: A neural network layer for persistence diagrams and new graph topological signatures, 2020.
 - [10] Y. Luo, K. Zhou, W. Zhang, Persistent homology enhanced graph attention network for molecular property prediction, *Neurocomputing* 527 (2023) 309–322. doi:10.1016/j.neucom.2022.11.079.
 - [11] T. Tekdogan, Analyzing the traffic of manets using graph neural networks, in: 2022 IEEE International Conference on Machine Learning and Applied Network Technologies (ICMLANT), 2022, pp. 1–4. doi:10.1109/ICMLANT56191.2022.9996518.
 - [12] H. Hu, M. J. Lee, Graph neural network-based clustering enhancement in vanet for cooperative driving, in: 2022 International Conference on Artificial Intelligence in Information and Communication (ICAIIIC), 2022, pp. 162–167. doi:10.1109/ICAIIIC54071.2022.9722625.
 - [13] F. Pasandideh, J. P. J. d. Costa, R. Kunst, W. Hardjawana, E. P. de Freitas, A systematic literature review of flying ad hoc networks: State-of-the-art, challenges, and perspectives, *Journal of Field Robotics* 40 (4) (2023) 955–979. arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1002/rob.22157>, doi:10.1002/rob.22157.
URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/rob.22157>
 - [14] W. Jiang, H. Han, Y. Zhang, J. Wang, M. He, W. Gu, J. Mu, X. Cheng, Graph neural networks for routing optimization: Challenges and opportunities, *Sustainability* 16 (21) (2024). doi:10.3390/su16219239.
URL <https://www.mdpi.com/2071-1050/16/21/9239>
 - [15] Y. Lu, W. Wen, P. Ren, et al., Uav ad hoc network routing algorithms in space-air-ground integrated networks: Challenges and directions, *Drones* 7 (7) (2023) 448. doi:10.3390/drones7070448.

- [16] J. Song, J. Li, Q. Wang, Y. Gu, T. Shen, Gnnppor: A proximal policy optimization multi-factor joint routing approach based on graph neural networks in fanets, *IEEE Networking Letters* 7 (2) (2025) 130–134. doi:10.1109/LNET.2025.3542762.
- [17] M. Nemati, B. Al Homssi, S. Krishnan, et al., Non-terrestrial networks with uavs: A projection on flying ad-hoc networks, *Drones* 6 (11) (2022) 334. doi:10.3390/drones6110334.
- [18] Z. Ye, K. Wang, Y. Chen, X. Jiang, G. Song, Multi-uav navigation for partially observable communication coverage by graph reinforcement learning, *IEEE Transactions on Mobile Computing* 22 (7) (2023) 4056–4069. doi:10.1109/TMC.2022.3146881.
- [19] Q. Ren, Y. Wang, H. Liu, Y. Dai, W. Ye, Y. Tang, Collaborative multi-target-tracking via graph-based deep reinforcement learning in uav swarm networks, *Ad Hoc Networks* 172 (2025) 103801. doi:10.1016/j.adhoc.2025.103801.
URL <https://www.sciencedirect.com/science/article/pii/S1570870525000496>
- [20] M. Saadati, S. M. Mazinani, A. A. Khazaei, S. J. S. M. Chabok, Energy efficient clustering for dense wireless sensor network by applying graph neural networks with coverage metrics, *Ad Hoc Networks* 156 (2024) 103432. doi:10.1016/j.adhoc.2024.103432.
URL <https://www.sciencedirect.com/science/article/pii/S157087052400043X>
- [21] M. N. Azadani, A. Boukerche, Stag: A novel interaction-aware path prediction method based on spatio-temporal attention graphs for connected automated vehicles, *Ad Hoc Networks* 138 (2023) 103021. doi:10.1016/j.adhoc.2022.103021.
URL <https://www.sciencedirect.com/science/article/pii/S1570870522001937>
- [22] C. G. Capanema, G. S. de Oliveira, F. A. Silva, T. R. Silva, A. A. Loureiro, Combining recurrent and graph neural networks to predict the next place's category, *Ad Hoc Networks* 138 (2023) 103016. doi:10.1016/j.adhoc.2022.103016.

URL <https://www.sciencedirect.com/science/article/pii/S1570870522001883>

- [23] Q. Li, Z. Han, X.-M. Wu, Deeper insights into graph convolutional networks for semi-supervised learning, in: Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence, AAAI'18/IAAI'18/EAAI'18, AAAI Press, 2018.
- [24] C. I. Kanatsoulis, A. Ribeiro, Graph neural networks are more powerful than we think, in: ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2024, pp. 7550–7554. doi:10.1109/ICASSP48485.2024.10447704.
- [25] J. Arevalo, T. Solorio, M. Montes-y Gómez, F. A. González, Gated multimodal networks, *Neural Comput. Appl.* 32 (14) (2020) 10209–10228. doi:10.1007/s00521-019-04559-1.
URL <https://doi.org/10.1007/s00521-019-04559-1>
- [26] Y.-H. H. Tsai, S. Bai, P. P. Liang, J. Z. Kolter, L.-P. Morency, R. Salakhutdinov, Multimodal transformer for unaligned multimodal language sequences, in: A. Korhonen, D. Traum, L. Màrquez (Eds.), Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Florence, Italy, 2019, pp. 6558–6569. doi:10.18653/v1/P19-1656.
URL <https://aclanthology.org/P19-1656/>
- [27] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization (2017). arXiv:1412.6980.
URL <https://arxiv.org/abs/1412.6980>